C++ Standard Library Immediate Issues to be moved in Kona, Nov. 2025

Doc. no. P3906R0

Date: 2025-11-07

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Immediate Issues

3343. Ordering of calls to `unlock()` and `notify_all()` in *Effects* element of `notify_all_at_thread_exit()` should be reversed

Section: 32.7.3 [thread.condition.nonmember] **Status:** Immediate **Submitter:** Lewis Baker **Opened:** 2019-11-21 **Last modified:** 2025-11-06

Priority: 3

Discussion:

32.7.3 [thread.condition.nonmember] p2 states:

> *Effects:* Transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed. This notification shall be as if:

```
    lk.unlock();
    cond.notify_all();
```

One common use-cases for the `notify_all_at_thread_exit()` is in conjunction with `thread::detach()` to allow detached threads to signal when they complete and to allow another thread to wait for them to complete using the `condition_variable/mutex` pair.

However, the current wording for `notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk)` makes it impossible to know when it is safe to destroy the `condition_variable` in the presence of spurious wake-ups and detached threads.

For example: Consider the following code-snippet:

```
#include <condition_variable>
#include <mutex>
#include <thread>

int main() {
  std::condition_variable cv;
  std::mutex mut;
  bool complete = false;

  std::thread{[&] {
    // do work here

    // Signal thread completion
    std::unique_lock lk{mut};
    complete = true;
    std::notify_all_at_thread_exit(cv, std::move(lk));
  }}.detach();

  // Wait until thread completes
  std::unique_lock lk{mut};
  cv.wait(lk, [&] { return complete; });

  // condition_variable destroyed on scope exit
  return 0;
}
```

This seems to an intended usage of `thread::detach()` and `std::notify_all_at_thread_exit()` and yet this code contains a race involving the call to `cv.notify_all()` on the created thread, and the destructor of the `condition_variable`.

To highlight the issue, consider the following case:

Let `T0` be the thread that executes `main()` and `T1` be the thread created by the `std::thread` construction.

`T0`: creates thread `T1`
`T0`: context-switched out by OS
`T1`: starts running

`T1`: acquires mutex lock
`T1`: sets `complete = true`

`T1`: calls `notify_all_at_thread_exit()`
`T1`: returns from thread-main function and runs all thread-local destructors
`T1`: calls `lk.unlock()`
`T1`: context-switched out by OS
`T0`: resumes execution
`T0`: acquires mutex lock
`T0`: calls `cv.wait()` which returns immediately as `complete` is `true`

`T0`: returns from `main()`, destroying `condition_variable`
`T1`: resumes execution

`T1`: calls `cv.notify_all()` on a dangling `cv` reference (undefined behaviour)

Other sequencings are possible involving spurious wake-ups of the `cv.wait()` call.

A proof-of-concept showing this issue can be found here.

The current wording requires releasing the mutex lock before calling `cv.notify_all()`. In the presence of spurious wake-ups of a `condition_variable::wait()`, there is no way to know whether or not a detached thread that called `std::notify_all_at_thread_exit()` has finished calling `cv.notify_all()`. This means there is no portable way to know when it will be safe for the waiting thread to destroy that `condition_variable`.

However, if we were to reverse the order of the calls to `lk.unlock()` and `cond.notify_all()` then the thread waiting for the detached thread to exit would not be able to observe the completion of the thread (in the above case, this would be observing the assignment of `true` to the `complete` variable) until the mutex lock was released by that thread and subsequently acquired by the waiting thread which would only happen after the completion of the call to `cv.notify_all()`. This would allow the above code example to eliminate the race between a subsequent destruction of the condition-variable and the call to `cv.notify_all()`.

[2019-12-08 Issue Prioritization]

Priority to 3 after reflector discussion.

[2019-12-15; Daniel synchronizes wording with N4842]

[2020-02, Prague]

Response from SG1: "We discussed it in Prague. We agree it's an error and SG1 agreed with the PR."

Previous resolution [SUPERSEDED]:

This wording is relative to N4842.

1. Change 32.7.3 [thread.condition.nonmember] as indicated:

```
void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);
```

[…]

-2- *Effects:* Transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed. This notification is equivalent to:

```
~~lk.unlock();~~
cond.notify_all();
lk.unlock();
```

[2023-06-13, Varna; Tim provides improved wording]

Addressed mailing list comments. Ask SG1 to check.

[Kona 2025-11-05; SG1 unanimously approved the proposed resolution.]

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N4950.

1. Change 32.7.3 [thread.condition.nonmember] as indicated:

```
void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);
```

[…]

-2- *Effects:* Transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits~~.~~ This notification is sequenced after all objects of thread storage duration associated with the current thread have been destroyed~~. This notification~~ and is equivalent to:

```
~~lk.unlock();~~
cond.notify_all();
lk.unlock();
```

3454. `pointer_traits::pointer_to` should be `constexpr`

Section: 20.2.3 [pointer.traits] **Status:** Immediate **Submitter:** Alisdair Meredith **Opened:** 2020-06-21 **Last modified:** 2025-11-05

Priority: 4

View all other issues in [pointer.traits].

Discussion:

Trying to implement a `constexpr std::list` (inspired by Tim Song's note on using variant members in the node) as part of evaluating the `constexpr` container and adapters proposals, I hit problems I could not code around in `pointer_traits`, as only the specialization for native pointers has a `constexpr pointer_to` function.

This means that containers of my custom allocator, that delegates all allocation behavior to `std::allocator<T>` but adds extra telemetry and uses a fancy pointer, does not work with the approach I tried for implementing `list` (common link type, shared between nodes, and stored as `end` sentinel directly in the `list` object).

[2020-07-17; Forwarded to LEWG after review in telecon]

[2022-07-19; Casey Carter comments]

This is no longer simply a theoretical problem that impedes implementing `constexpr std::list`, but an actual defect affecting current implementations of `constexpr std::string`. More specifically, it makes it impossible to support so-called "fancy pointers" in a `constexpr basic_string` that performs the small string optimization (SSO). (`pointer_traits::pointer_to` is critically necessary to get a pointer that designates the SSO buffer.) As things currently stand, `constexpr basic_string` can support fancy pointers *or* SSO, but not both.

[Wrocław 2024-11-18; LEWG approves the direction]

Should there be an Annex C entry noting that program-defined specializations need to add `constexpr` to be conforming?

[2025-10-20; Set priority to 4 based on age of issue and lack of recent interest.]

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N4861.

1. Modify 20.2.3 [pointer.traits] as indicated:

-1- The class template `pointer_traits` supplies a uniform interface to certain attributes of pointer-like types.

```
namespace std {
  template<class Ptr> struct pointer_traits {
    using pointer         = Ptr;
    using element_type    = see below;
    using difference_type = see below;

    template<class U> using rebind = see below;

    static constexpr pointer pointer_to(see below r);
  };
  […]
}
```

2. Modify 20.2.3.3 [pointer.traits.functions] as indicated:

```
static constexpr pointer pointer_traits::pointer_to(see below r);
static constexpr pointer pointer_traits<T*>::pointer_to(see below r) noexcept;
```

-1- *Mandates:* For the first member function, `Ptr::pointer_to(r)` is well-formed.

-2- *Preconditions:* For the first member function, `Ptr::pointer_to(r)` returns a pointer to `r` through which indirection is valid.

-3- *Returns:* The first member function returns `Ptr::pointer_to(r)`. The second member function returns `addressof(r)`.

-4- *Remarks:* If `element_type` is *cv* `void`, the type of `r` is unspecified; otherwise, it is `element_type&`.

4015. LWG 3973 broke `const` overloads of `std::optional` monadic operations

Section: 22.5.3.8 [optional.monadic] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2023-11-24 **Last modified:** 2025-11-05

Priority: 1

Discussion:

The resolution of LWG 3973[(i)] (adopted in Kona) changed all occurrences of `value()` to `*val`. The intention was not to change the meaning, just avoid the non-freestanding `value()` function, and avoid ADL that would be caused by using `**this`. However, in the `const` overloads such as `and_then(F&&) const` the type of `value()` was `const T&`, but the type of `*val` is always `T&`. This implies that the const overloads invoke the callable with a non-const argument, which is incorrect (and would be undefined behaviour for a `const std::optional<T>`).

On the LWG reflector it was suggested that we should rewrite the specification of `std::optional` to stop using an exposition-only data member of type `T*`. No such member ever exists in real implemetations, so it is misleading and leads to specification bugs of this sort.

Change the class definition in 22.5.3.1 [optional.optional.general] to use a union, and update every use of `val` accordingly throughout 22.5.3 [optional.optional]. For consistency with 22.8.6.1 [expected.object.general] we might also want to introduce a `bool has_val` member and refer to that in the specification.

```
private:
  T *val;         // exposition only
  bool has_val;   // exposition only
  union {
    T val;        // exposition only
  };
};
```

For example, in 22.5.3.9 [optional.mod]:

-1- *Effects*: If `*this` contains a value, calls `val->T::~T()` to destroy the contained value and sets `has_val` to `false`; otherwise no effect.

[2023-11-26; Daniel provides wording]

The proposed wording is considerably influenced by that of the specification of `expected`, but attempts to reduce the amount of changes to not perfectly mimic it. Although "the contained value" is a magic word of power it seemed feasible and simpler to use the new exposition-only member *`val`* directly in some (but not all) places, usually involved with initializations.

Furthermore, I have only added "and sets *`has_val`* to `true`/`false`" where either the *Effects* wording says "otherwise no effect" or in other cases if the postconditions did not already say that indirectly. I also added extra mentioning of *`has_val`* changes in tables where different cells had very different effects on that member (unless these cells specify postconditions), to prevent misunderstanding.

[2024-03-11; Reflector poll]

Set priority to 1 after reflector poll in November 2023. Six votes for 'Tentatively Ready' but enough uncertainty to deserve discussion at a meeting.

Previous resolution [SUPERSEDED]:

This wording is relative to N4964 *after* application of the wording of LWG 3973[(i)].

1. Modify 22.5.3.1 [optional.optional.general], class template `optional` synopsis, as indicated:

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type = T;
    […]
  private:
    bool has_val; // exposition only
```

```
    union {
      T val*val; // exposition only
    };
  };

  […]
}
```

2. Modify 22.5.3.1 [optional.optional.general] as indicated:

> -2- Member *has_val* indicates whether an `optional<T>` object contains a value~~When an `optional<T>` object contains a value, member `val` points to the contained value~~.

3. Modify 22.5.3.2 [optional.ctor] as indicated:

> [*Drafting note*: Normatively, this subclause doesn't require any changes, but I'm suggesting to replace phrases of the form "[…]initializes the contained value with"] by "[…]initializes *val* with" as we do in 22.8.6.2 [expected.object.cons]. I intentionally did not add extra "and sets *has_val* to `true/false`" since those effects are already guaranteed by the postconditions]

```
constexpr optional(const optional& rhs);
```

> -4- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with `*rhs.val`.
>
> -5- *Postconditions*: `rhs.has_value() == this->has_value()`.
>
> […]

```
constexpr optional(optional&& rhs) noexcept(see below);
```

> -8- *Constraints*: […]
>
> -9- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with `std::move(*rhs.val)`. `rhs.has_value()` is unchanged.
>
> -10- *Postconditions*: `rhs.has_value() == this->has_value()`.
>
> […]

```
template<class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

> -13- *Constraints*: […]
>
> -14- *Effects*: Direct-non-list-initializes *val*~~the contained value~~ with `std::forward<Args>(args)...`.
>
> -15- *Postconditions*: `*this` contains a value.
>
> […]

```
template<class U, class... Args>
  constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

> -18- *Constraints*: […]
>
> -19- *Effects*: Direct-non-list-initializes *val*~~the contained value~~ with `il, std::forward<Args>(args)...`.
>
> -20- *Postconditions*: `*this` contains a value.
>
> […]

```
template<class U = T> constexpr explicit(see below) optional(U&& v);
```

> -23- *Constraints*: […]
>
> -24- *Effects*: Direct-non-list-initializes *val*~~the contained value~~ with `std::forward<U>(v)`.
>
> -25- *Postconditions*: `*this` contains a value.
>
> […]

```
template<class U> constexpr explicit(see below) optional(const optional<U>& rhs);
```

> -28- *Constraints*: […]
>
> -29- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with `*rhs.val`.
>
> -30- *Postconditions*: `rhs.has_value() == this->has_value()`.
>
> […]

```
template<class U> constexpr explicit(see below) optional(optional<U>&& rhs);
```

> -33- *Constraints*: […]
>
> -34- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with `std::move(*rhs.val)`. `rhs.has_value()` is unchanged.

-35- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

4. Modify 22.5.3.3 [optional.dtor] as indicated:

```
constexpr ~optional();
```

-1- *Effects*: If `is_trivially_destructible_v<T> != true` and `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()`~~`.`~~.

5. Modify 22.5.3.4 [optional.assign] as indicated:

```
constexpr optional<T>& operator=(nullopt_t) noexcept;
```

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()` to destroy the contained value and sets *`has_val`* to `false`; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

```
constexpr optional<T>& operator=(const optional& rhs);
```

-4- *Effects*: See Table 58.

Table 58 — `optional::operator=(const optional&)` effects [tab:optional.assign.copy]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~`*`~~`rhs`*`.val`* to *`val`*~~the contained value~~	direct-non-list-initializes *`val`*~~the contained value~~ with ~~`*`~~`rhs`*`.val`* and sets *`has_val`* to `true`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()` and sets *`has_val`* to `false`	no effect

-5- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

```
constexpr optional<T>& operator=(optional&& rhs) noexcept(see below);
```

-8- *Constraints*: […]

-9- *Effects*: See Table 59. The result of the expression `rhs.has_value()` remains unchanged.

-10- *Postconditions*: `rhs.has_value() == this->has_value()`.

-11- *Returns*: `*this`.

Table 59 — `optional::operator=(optional&&)` effects [tab:optional.assign.move]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `std::move(`~~`*`~~`rhs`*`.val`*`)` to *`val`*~~the contained value~~	direct-non-list-initializes *`val`*~~the contained value~~ with `std::move(`~~`*`~~`rhs`*`.val`*`)` and sets *`has_val`* to `true`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()` and sets *`has_val`* to `false`	no effect

-12- *Remarks*: […]

-13- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of ~~`*`~~`rhs`*`.val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of ~~`*`~~*`val`*~~`val`~~ and ~~`*`~~`rhs`*`.val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s move assignment.

```
template<class U = T> constexpr optional<T>& operator=(U&& v);
```

-14- *Constraints*: […]

-15- *Effects*: If `*this` contains a value, assigns `std::forward<U>(v)` to *`val`*~~the contained value~~; otherwise direct-non-list-initializes *`val`*~~the contained value~~ with `std::forward<U>(v)`.

-16- *Postconditions*: `*this` contains a value.

-17- *Returns*: `*this`.

-18- *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of *`val`*~~`*val`~~ and `v` is determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(const optional<U>& rhs);
```

-19- *Constraints*: […]

-20- *Effects*: See Table 60.

Table 60 — `optional::operator=(const optional<U>&)` effects [tab:optional.assign.copy.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `*rhs.val` to `val`~~the contained value~~	direct-non-list-initializes `val`~~the contained value~~ with `*rhs.val` and sets *`has_val`* to `true`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~`val.T::~T()` and sets *`has_val`* to `false`	no effect

-21- *Postconditions*: `rhs.has_value() == this->has_value()`.

-22- *Returns*: `*this`.

-23- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val`~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `val`~~`*val`~~ and `*rhs.val`~~`val`~~ is determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
```

-24- *Constraints*: […]

-25- *Effects*: See Table 61. The result of the expression `rhs.has_value()` remains unchanged.

Table 61 — `optional::operator=(optional<U>&&)` effects [tab:optional.assign.move.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `std::move(*rhs.val)` to `val`~~the contained value~~	direct-non-list-initializes `val`~~the contained value~~ with `std::move(*rhs.val)` and sets *`has_val`* to `true`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~`val.T::~T()` and sets *`has_val`* to `false`	no effect

-26- *Postconditions*: `rhs.has_value() == this->has_value()`.

-27- *Returns*: `*this`.

-28- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val`~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `val`~~`*val`~~ and `*rhs.val`~~`val`~~ is determined by the exception safety guarantee of `T`'s assignment.

```
template<class... Args> constexpr T& emplace(Args&&... args);
```

-29- *Mandates*: […]

-30- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes `val`~~the contained value~~ with `std::forward<Args>(args)...`.

-31- *Postconditions*: `*this` contains a value.

-32- *Returns*: `val`~~A reference to the new contained value~~.

[…]

-34- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `val`~~`*val`~~ (if any) has been destroyed.

```
template<class U, class... Args> constexpr T& emplace(initializer_list<U> il, Args&&... args);
```

-35- *Constraints*: […]

-36- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes `val`~~the contained value~~ with `il, std::forward<Args>(args)...`.

-37- *Postconditions*: `*this` contains a value.

-38- *Returns*: `val`~~A reference to the new contained value~~.

[…]

-40- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `val`~~`*val`~~ (if any) has been destroyed.

6. Modify 22.5.3.5 [optional.swap] as indicated:

```
constexpr void swap(optional& rhs) noexcept(see below);
```

-1- *Mandates*: […]

-2- *Preconditions*: […]

-3- *Effects*: See Table 62.

Table 62 — `optional::swap(optional&)` effects [tab:optional.swap]

	`*this` contains a value	`*this` does not contain a value
`rhs` contains a value	calls `swap(val*(*this), *rhs.val)`	direct-non-list-initializes `val`~~the contained value of `*this`~~ with `std::move(*rhs.val)`, followed by `rhs.val.`~~`val->`~~`T::~T();` postcondition is that `*this` contains a value and `rhs` does not contain a value
`rhs` does not contain a value	direct-non-list-initializes ~~the contained value of~~ `rhs.val` with `std::move(val`~~`*(*this)`~~`)`, followed by `val.`~~`val->`~~`T::~T();` postcondition is that `*this` does not contain a value and `rhs` contains a value	no effect

-4- *Throws*: […]

-5- *Remarks*: […]

-6- If any exception is thrown, the results of the expressions `this->has_value()` and `rhs.has_value()` remain unchanged. If an exception is thrown during the call to function `swap`, the state of `val`~~`*val`~~ and ~~`*`~~`rhs.val`~~`val`~~ is determined by the exception safety guarantee of `swap` for lvalues of `T`. If an exception is thrown during the call to `T`'s move constructor, the state of `val`~~`*val`~~ and ~~`*`~~`rhs.val`~~`val`~~ is determined by the exception safety guarantee of `T`'s move constructor.

7. Modify 22.5.3.7 [optional.observe] as indicated:

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

-1- *Preconditions*: `*this` contains a value.

-2- *Returns*: `addressof(val)`~~`val`~~.

-3- […]

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

-4- *Preconditions*: `*this` contains a value.

-5- *Returns*: `val`~~`*val`~~.

-6- […]

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

-7- *Preconditions*: `*this` contains a value.

-8- *Effects*: Equivalent to: `return std::move(val`~~`*val`~~`);`

```
constexpr explicit operator bool() const noexcept;
```

~~-9- *Returns*: `true` if and only if `*this` contains a value.~~

~~-10- *Remarks*: This function is a constexpr function.~~

```
constexpr bool has_value() const noexcept;
```

-11- *Returns*: `has_val`~~`true if and only if *this contains a value`~~.

-12- *Remarks*: These functions are~~This function is a~~ constexpr functions.

```
constexpr const T& value() const &;
constexpr T& value() &;
```

-13- *Effects*: Equivalent to:

```
return has_value() ? val~~*val~~ : throw bad_optional_access();
```

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```

-14- *Effects*: Equivalent to:

```
return has_value() ? std::move(val~~*val~~) : throw bad_optional_access();
```

```
template<class U> constexpr T value_or(U&& v) const &;
```

-15- *Mandates*: […]

-16- *Effects*: Equivalent to:

```
return has_value() ? val~~*this~~ : static_cast<T>(std::forward<U>(v));
```

```
template<class U> constexpr T value_or(U&& v) &&;
```

-17- *Mandates*: […]

-18- *Effects*: Equivalent to:

```
return has_value() ? std::move(val~~*this~~) : static_cast<T>(std::forward<U>(v));
```

8. Modify 22.5.3.8 [optional.monadic] as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let `U` be `invoke_result_t<F, decltype((val)~~*val~~)>`.

-2- *Mandates*: […]

-3- *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), val~~*val~~);
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

-4- Let `U` be `invoke_result_t<F, decltype(std::move(val~~*val~~))>`.

-5- *Mandates*: […]

-6- *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), std::move(val~~*val~~));
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

-7- Let `U` be `remove_cv_t<invoke_result_t<F, decltype((val)~~*val~~)>>`.

-8- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), val~~*val~~));
```

is well-formed for some invented variable `u`.

[…]

-9- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), val~~*val~~)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

-10- Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(val~~*val~~))>>`.

-11- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), std::move(val~~*val~~)));
```

is well-formed for some invented variable `u`.

[…]

-12- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(val~~*val~~))`; otherwise, `optional<U>()`.

9. Modify 22.5.3.9 [optional.mod] as indicated:

```
constexpr void reset() noexcept;
```

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~`val.T::~T()` to destroy the contained value and sets *has_val* to `false`; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

[St. Louis 2024-06-24; Jonathan provides improved wording]

[2024-08-21; LWG telecon]

During telecon review it was suggested to replace 22.5.3.1 [optional.optional.general] p1 and p2. On the reflector Daniel requested to keep the "additional storage" prohibition, so that will be addressed by issue 4141[(i)] instead.

[2024-10-02; Jonathan tweaks proposed resolution]

On the reflector we decided that the union member should use `remove_cv_t`, as proposed for `expected` by issue 3891[(i)]. The rest of the proposed resolution is unchanged, so that edit was made in-place below, instead of as a new resolution that supersedes the old one.

Previous resolution [SUPERSEDED]:

This wording is relative to N4988.

1. Modify 22.5.3.1 [optional.optional.general], class template `optional` synopsis, as indicated:

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type = T;
    […]
  private:
    ~~*val // exposition only~~;
    union {
      remove_cv_t<T> val; // exposition only
    };
  };

  […]
}
```

2. Modify 22.5.3.1 [optional.optional.general] as indicated:

-1- When its member *`val`* is active (11.5.1 [class.union.general]), an instance of `optional<T>` is said to *contain a value*, and *`val`* is referred to as its *contained value*. ~~Any instance of `optional<T>` at any given time either contains a value or does not contain a value. When an instance of `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the~~ An optional object's contained value ~~*contained value*,~~ is allocated within the storage of the optional object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. ~~When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.~~

~~-2- When an `optional<T>` object contains a value, member `val` points to the contained value.~~

3. Modify 22.5.3.2 [optional.ctor] as indicated:

```
constexpr optional(const optional& rhs);
```

-4- *Effects*: If `rhs` contains a value, direct-non-list-initializes *`val`*~~the contained value~~ with ~~`*`~~`rhs.`*`val`*.

-5- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

```
constexpr optional(optional&& rhs) noexcept(see below);
```

-8- *Constraints*: […]

-9- *Effects*: If `rhs` contains a value, direct-non-list-initializes *`val`*~~the contained value~~ with `std::move(`~~`*`~~`rhs.`*`val`*`)`. `rhs.has_value()` is unchanged.

-10- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

```
template<class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

-13- *Constraints*: […]

-14- *Effects*: Direct-non-list-initializes *`val`*~~the contained value~~ with `std::forward<Args>(args)...`.

-15- *Postconditions*: `*this` contains a value.

[…]

```
template<class U, class... Args>
  constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

-18- *Constraints*: […]

-19- *Effects*: Direct-non-list-initializes *`val`*~~the contained value~~ with `il, std::forward<Args>(args)...`.

-20- *Postconditions*: `*this` contains a value.

[…]

```
template<class U = T> constexpr explicit(see below) optional(U&& v);
```

-23- *Constraints*: […]

-24- *Effects*: Direct-non-list-initializes *`val`*~~the contained value~~ with `std::forward<U>(v)`.

-25- *Postconditions*: `*this` contains a value.

[…]

`template<class U> constexpr explicit(`*see below*`) optional(const optional<U>& rhs);`

-28- *Constraints*: […]

-29- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`rhs.`*`val`*.

-30- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

`template<class U> constexpr explicit(`*see below*`) optional(optional<U>&& rhs);`

-33- *Constraints*: […]

-34- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with `std::move(`~~*~~`rhs.`*`val`*`)`. `rhs.has_value()` is unchanged.

-35- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

4. Modify 22.5.3.3 [optional.dtor] as indicated:

`constexpr ~optional();`

-1- *Effects*: If `is_trivially_destructible_v<T> != true` and `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()`.

5. Modify 22.5.3.4 [optional.assign] as indicated:

`constexpr optional<T>& operator=(nullopt_t) noexcept;`

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()` to destroy the contained value; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

`constexpr optional<T>& operator=(const optional& rhs);`

-4- *Effects*: See Table 58.

Table 58 — `optional::operator=(const optional&)` effects [tab:optional.assign.copy]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~*~~`rhs.`*`val`* to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`rhs.`*`val`*
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-5- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

`constexpr optional<T>& operator=(optional&& rhs) noexcept(`*see below*`);`

-8- *Constraints*: […]

-9- *Effects*: See Table 59. The result of the expression `rhs.has_value()` remains unchanged.

-10- *Postconditions*: `rhs.has_value() == this->has_value()`.

-11- *Returns*: `*this`.

Table 59 — `optional::operator=(optional&&)` effects [tab:optional.assign.move]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `std::move(`~~*~~`rhs.`*`val`*`)` to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with `std::move(`~~*~~`rhs.`*`val`*`)`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-12- *Remarks*: […]

-13- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of ~~*~~`rhs.`*`val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the states ~~state~~ of ~~*~~*`val`*~~`val`~~ and ~~*~~`rhs.`*`val`*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s move assignment.

`template<class U = T> constexpr optional<T>& operator=(U&& v);`

-14- *Constraints*: […]

-15- *Effects*: If `*this` contains a value, assigns `std::forward<U>(v)` to *val*~~the contained value~~; otherwise direct-non-list-initializes *val*~~the contained value~~ with `std::forward<U>(v)`.

-16- *Postconditions*: `*this` contains a value.

-17- *Returns*: `*this`.

-18- *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~`*val`~~ and `v` are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(const optional<U>& rhs);
```

-19- *Constraints*: […]

-20- *Effects*: See Table 60.

Table 60 — `optional::operator=(const optional<U>&)` effects [tab:optional.assign.copy.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~`*`~~`rhs.`*`val`* to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with ~~`*`~~`rhs.`*`val`*
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-21- *Postconditions*: `rhs.has_value() == this->has_value()`.

-22- *Returns*: `*this`.

-23- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of ~~`*`~~`rhs.`*`val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~`*val`~~ and ~~`*`~~`rhs.`*`val`*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
```

-24- *Constraints*: […]

-25- *Effects*: See Table 61. The result of the expression `rhs.has_value()` remains unchanged.

Table 61 — `optional::operator=(optional<U>&&)` effects [tab:optional.assign.move.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `std::move(`~~`*`~~`rhs.`*`val`*`)` to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with `std::move(`~~`*`~~`rhs.`*`val`*`)`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-26- *Postconditions*: `rhs.has_value() == this->has_value()`.

-27- *Returns*: `*this`.

-28- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of ~~`*`~~`rhs.`*`val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~`*val`~~ and ~~`*`~~`rhs.`*`val`*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class... Args> constexpr T& emplace(Args&&... args);
```

-29- *Mandates*: […]

-30- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes *val*~~the contained value~~ with `std::forward<Args>(args)...`.

-31- *Postconditions*: `*this` contains a value.

-32- *Returns*: *val*~~A reference to the new contained value~~.

[…]

-34- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous *val*~~`*val`~~ (if any) has been destroyed.

```
template<class U, class... Args> constexpr T& emplace(initializer_list<U> il, Args&&... args);
```

-35- *Constraints*: […]

-36- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes *val*~~the contained value~~ with `il, std::forward<Args>(args)...`.

-37- *Postconditions*: `*this` contains a value.

-38- *Returns*: *val*~~A reference to the new contained value~~.

[…]

-40- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous *val*~~`*val`~~ (if any) has been destroyed.

6. Modify 22.5.3.5 [optional.swap] as indicated:

```
constexpr void swap(optional& rhs) noexcept(see below);
```

-1- *Mandates*: […]

-2- *Preconditions*: […]

-3- *Effects*: See Table 62.

Table 62 — `optional::swap(optional&)` effects [tab:optional.swap]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	calls `swap(val~~*(*this)~~, *rhs.val)`	direct-non-list-initializes `val`~~the contained value of `*this~~ with `std::move(*rhs.val)`, followed by `rhs.val.~~val ->~~T::~T();` postcondition is that `*this` contains a value and `rhs` does not contain a value
`rhs` does not contain a value	direct-non-list-initializes ~~the contained value of~~ `rhs.val` with `std::move(val~~*(*this)~~)`, followed by `val.~~val ->~~T::~T();` postcondition is that `*this` does not contain a value and `rhs` contains a value	no effect

-4- *Throws*: […]

-5- *Remarks*: […]

-6- If any exception is thrown, the results of the expressions `this->has_value()` and `rhs.has_value()` remain unchanged. If an exception is thrown during the call to function `swap`, the state of `val`~~`*val`~~ and ~~`*`~~`rhs.val`~~`val`~~ is determined by the exception safety guarantee of `swap` for lvalues of `T`. If an exception is thrown during the call to `T`'s move constructor, the states ~~state~~ of `val`~~`*val`~~ and ~~`*`~~`rhs.val`~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s move constructor.

7. Modify 22.5.3.7 [optional.observe] as indicated:

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

-1- *Preconditions*: `*this` contains a value.

-2- *Returns*: `addressof(val)`~~`val`~~.

-3- […]

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

-4- *Preconditions*: `*this` contains a value.

-5- *Returns*: `val`~~`*val`~~.

-6- […]

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

-7- *Preconditions*: `*this` contains a value.

-8- *Effects*: Equivalent to: `return std::move(val~~*val~~);`

```
constexpr explicit operator bool() const noexcept;
```

-9- *Returns*: `true` if and only if `*this` contains a value.

-10- *Remarks*: This function is a constexpr function.

```
constexpr bool has_value() const noexcept;
```

-11- *Returns*: `true` if and only if `*this` contains a value.

-12- *Remarks*: This function is a constexpr function.

```
constexpr const T& value() const &;
constexpr T& value() &;
```

-13- *Effects*: Equivalent to:

```
    return has_value() ? val~~*val~~ : throw bad_optional_access();
```

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```

-14- *Effects*: Equivalent to:

```
            return has_value() ? std::move(val~~*val~~) : throw bad_optional_access();

    template<class U> constexpr T value_or(U&& v) const &;
```

-15- *Mandates*: […]

-16- *Effects*: Equivalent to:

```
            return has_value() ? val~~*this~~ : static_cast<T>(std::forward<U>(v));

    template<class U> constexpr T value_or(U&& v) &&;
```

-17- *Mandates*: […]

-18- *Effects*: Equivalent to:

```
            return has_value() ? std::move(val~~*this~~) : static_cast<T>(std::forward<U>(v));
```

8. Modify 22.5.3.8 [optional.monadic] as indicated:

```
    template<class F> constexpr auto and_then(F&& f) &;
    template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let `U` be `invoke_result_t<F, decltype((val)~~*val~~)>`.

-2- *Mandates*: […]

-3- *Effects*: Equivalent to:

```
            if (*this) {
              return invoke(std::forward<F>(f), val~~*val~~);
            } else {
              return remove_cvref_t<U>();
            }

    template<class F> constexpr auto and_then(F&& f) &&;
    template<class F> constexpr auto and_then(F&& f) const &&;
```

-4- Let `U` be `invoke_result_t<F, decltype(std::move(val~~*val~~))>`.

-5- *Mandates*: […]

-6- *Effects*: Equivalent to:

```
            if (*this) {
              return invoke(std::forward<F>(f), std::move(val~~*val~~));
            } else {
              return remove_cvref_t<U>();
            }

    template<class F> constexpr auto transform(F&& f) &;
    template<class F> constexpr auto transform(F&& f) const &;
```

-7- Let `U` be `remove_cv_t<invoke_result_t<F, decltype((val)~~*val~~)>>`.

-8- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
            U u(invoke(std::forward<F>(f), val~~*val~~));
```

is well-formed for some invented variable `u`.

[…]

-9- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), val~~*val~~)`; otherwise, `optional<U>()`.

```
    template<class F> constexpr auto transform(F&& f) &&;
    template<class F> constexpr auto transform(F&& f) const &&;
```

-10- Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(val~~*val~~))>>`.

-11- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
            U u(invoke(std::forward<F>(f), std::move(val~~*val~~)));
```

is well-formed for some invented variable `u`.

[…]

-12- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(val~~*val~~))`; otherwise, `optional<U>()`.

9. Modify 22.5.3.9 [optional.mod] as indicated:

```
    constexpr void reset() noexcept;
```

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~`val.T::~T()` to destroy the contained value; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

[2025-11-03; Tomasz tweaks proposed resolution]

Updated converting constructor and assignments to use `operator*()` directly, required to correctly support `optional<T&>`. Also update corresponding constructor in specialization.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 22.5.3.1 [optional.optional.general], class template `optional` synopsis, as indicated:

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type = T;
    […]
  private:
    T* val; // exposition only
    union {
      remove_cv_t<T> val; // exposition only
    };
  };

  […]
}
```

2. Modify 22.5.3.1 [optional.optional.general] as indicated:

-1- An instance of `optional<T>` is said to *contain a value* when and only when its member *`val`* is active (11.5.1 [class.union.general]); *`val`* is referred to as its *contained value*. An object of type `optional<T>` at any given time either contains a value or does not contain a value. When an object of type `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the An optional object's contained value *contained value*, is nested within (6.8.2 [intro.object]) the optional object. When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

-2- When an `optional<T>` object contains a value, member `val` points to the contained value.

3. Modify 22.5.3.2 [optional.ctor] as indicated:

```
constexpr optional(const optional& rhs);
```

-4- *Effects*: If `rhs` contains a value, direct-non-list-initializes *`val`*the contained value with `*rhs.val`.

-5- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

```
constexpr optional(optional&& rhs) noexcept(see below);
```

-8- *Constraints*: […]

-9- *Effects*: If `rhs` contains a value, direct-non-list-initializes *`val`*the contained value with `std::move(*rhs.val)`. `rhs.has_value()` is unchanged.

-10- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

```
template<class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

-13- *Constraints*: […]

-14- *Effects*: Direct-non-list-initializes *`val`*the contained value with `std::forward<Args>(args)...`.

-15- *Postconditions*: `*this` contains a value.

[…]

```
template<class U, class... Args>
  constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

-18- *Constraints*: […]

-19- *Effects*: Direct-non-list-initializes *`val`*the contained value with `il, std::forward<Args>(args)...`.

-20- *Postconditions*: `*this` contains a value.

[…]

```
template<class U = T> constexpr explicit(see below) optional(U&& v);
```

-23- *Constraints*: […]

-24- *Effects*: Direct-non-list-initializes *`val`*the contained value with `std::forward<U>(v)`.

-25- *Postconditions*: `*this` contains a value.

[…]

`template<class U> constexpr explicit(`*see below*`) optional(const optional<U>& rhs);`

-28- *Constraints*: […]

-29- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`rhs.operator*()`.

-30- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

`template<class U> constexpr explicit(`*see below*`) optional(optional<U>&& rhs);`

-33- *Constraints*: […]

-34- *Effects*: If `rhs` contains a value, direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`std::move(rhs).operator*()`. `rhs.has_value()` is unchanged.

-35- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

4. Modify 22.5.3.3 [optional.dtor] as indicated:

`constexpr ~optional();`

-1- *Effects*: If `is_trivially_destructible_v<T> != true` and `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()`.

5. Modify 22.5.3.4 [optional.assign] as indicated:

`constexpr optional<T>& operator=(nullopt_t) noexcept;`

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()` to destroy the contained value; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

`constexpr optional<T>& operator=(const optional& rhs);`

-4- *Effects*: See Table 58.

Table 58 — `optional::operator=(const optional&)` effects [tab:optional.assign.copy]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~*~~`rhs.`*`val`* to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`rhs.`*`val`*
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-5- *Postconditions*: `rhs.has_value() == this->has_value()`.

[…]

`constexpr optional<T>& operator=(optional&& rhs) noexcept(`*see below*`);`

-8- *Constraints*: […]

-9- *Effects*: See Table 59. The result of the expression `rhs.has_value()` remains unchanged.

-10- *Postconditions*: `rhs.has_value() == this->has_value()`.

-11- *Returns*: `*this`.

Table 59 — `optional::operator=(optional&&)` effects [tab:optional.assign.move]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `std::move(`~~*~~`rhs.`*`val`*`)` to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with `std::move(`~~*~~`rhs.`*`val`*`)`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-12- *Remarks*: […]

-13- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of ~~*~~`rhs.`*`val`*~~`val`~~ is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the states ~~state~~ of ~~*~~*`val`*~~`val`~~ and ~~*~~`rhs.`*`val`*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s move assignment.

`template<class U = T> constexpr optional<T>& operator=(U&& v);`

-14- *Constraints*: […]

-15- *Effects*: If `*this` contains a value, assigns `std::forward<U>(v)` to *val*~~the contained value~~; otherwise direct-non-list-initializes *val*~~the contained value~~ with `std::forward<U>(v)`.

-16- *Postconditions*: `*this` contains a value.

-17- *Returns*: `*this`.

-18- *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~*val~~ and `v` are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(const optional<U>& rhs);
```

-19- *Constraints*: […]

-20- *Effects*: See Table 60.

Table 60 — `optional::operator=(const optional<U>&)` effects [tab:optional.assign.copy.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~*~~`rhs.operator*()` to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`rhs.operator*()`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-21- *Postconditions*: `rhs.has_value() == this->has_value()`.

-22- *Returns*: `*this`.

-23- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of ~~*~~`rhs.`*val*~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~*val~~ and ~~*~~`rhs.`*val*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
```

-24- *Constraints*: […]

-25- *Effects*: See Table 61. The result of the expression `rhs.has_value()` remains unchanged.

Table 61 — `optional::operator=(optional<U>&&)` effects [tab:optional.assign.move.templ]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns ~~*~~`std::move(rhs).operator*()` to *val*~~the contained value~~	direct-non-list-initializes *val*~~the contained value~~ with ~~*~~`std::move(rhs).operator*()`
`rhs` does not contain a value	destroys the contained value by calling ~~`val->`~~*`val.`*`T::~T()`	no effect

-26- *Postconditions*: `rhs.has_value() == this->has_value()`.

-27- *Returns*: `*this`.

-28- If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of ~~*~~`rhs.`*val*~~`val`~~ is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states ~~state~~ of *val*~~*val~~ and ~~*~~`rhs.`*val*~~`val`~~ are ~~is~~ determined by the exception safety guarantee of `T`'s assignment.

```
template<class... Args> constexpr T& emplace(Args&&... args);
```

-29- *Mandates*: […]

-30- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes *val*~~the contained value~~ with `std::forward<Args>(args)...`.

-31- *Postconditions*: `*this` contains a value.

-32- *Returns*: *val*~~A reference to the new contained value~~.

[…]

-34- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous *val*~~*val~~ (if any) has been destroyed.

```
template<class U, class... Args> constexpr T& emplace(initializer_list<U> il, Args&&... args);
```

-35- *Constraints*: […]

-36- *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes *val*~~the contained value~~ with `il, std::forward<Args>(args)...`.

-37- *Postconditions*: `*this` contains a value.

-38- *Returns*: *val*~~A reference to the new contained value~~.

[…]

-40- *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous *val*~~*val~~ (if any) has been destroyed.

6. Modify 22.5.3.5 [optional.swap] as indicated:

```
constexpr void swap(optional& rhs) noexcept(see below);
```

-1- *Mandates*: […]

-2- *Preconditions*: […]

-3- *Effects*: See Table 62.

Table 62 — `optional::swap(optional&)` effects [tab:optional.swap]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	calls `swap(val~~*(*this)~~, *rhs.val)`	direct-non-list-initializes *val*~~the contained value of *this~~ with `std::move(*rhs.val)`, followed by `rhs.val.~~val->~~T::~T();` postcondition is that `*this` contains a value and `rhs` does not contain a value
`rhs` does not contain a value	direct-non-list-initializes ~~the contained value of~~ `rhs.val` with `std::move(val~~*(*this)~~)`, followed by *val*`.~~val->~~T::~T();` postcondition is that `*this` does not contain a value and `rhs` contains a value	no effect

-4- *Throws*: […]

-5- *Remarks*: […]

-6- If any exception is thrown, the results of the expressions `this->has_value()` and `rhs.has_value()` remain unchanged. If an exception is thrown during the call to function `swap`, the state of *val*~~*val~~ and ~~*~~`rhs.`*val*~~val~~ is determined by the exception safety guarantee of `swap` for lvalues of `T`. If an exception is thrown during the call to `T`'s move constructor, the states ~~state~~ of *val*~~*val~~ and ~~*~~`rhs.`*val*~~val~~ are ~~is~~ determined by the exception safety guarantee of `T`'s move constructor.

7. Modify 22.5.3.7 [optional.observe] as indicated:

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

-1- *Preconditions*: `*this` contains a value.

-2- *Returns*: `addressof(`*val*`)`~~*val~~.

-3- […]

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

-4- *Preconditions*: `*this` contains a value.

-5- *Returns*: *val*~~*val~~.

-6- […]

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

-7- *Preconditions*: `*this` contains a value.

-8- *Effects*: Equivalent to: `return std::move(`*val*~~*val~~`);`

```
constexpr explicit operator bool() const noexcept;
```

-9- *Returns*: `true` if and only if `*this` contains a value.

-10- *Remarks*: This function is a constexpr function.

```
constexpr bool has_value() const noexcept;
```

-11- *Returns*: `true` if and only if `*this` contains a value.

-12- *Remarks*: This function is a constexpr function.

```
constexpr const T& value() const &;
constexpr T& value() &;
```

-13- *Effects*: Equivalent to:

```
return has_value() ? val~~*val~~ : throw bad_optional_access();
```

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```

-14- *Effects*: Equivalent to:

```
return has_value() ? std::move(val~~*val~~) : throw bad_optional_access();
```

```
template<class U> constexpr T value_or(U&& v) const &;
```

-15- *Mandates*: […]

-16- *Effects*: Equivalent to:

```
return has_value() ? val~~*this~~ : static_cast<T>(std::forward<U>(v));
```

```
template<class U> constexpr T value_or(U&& v) &&;
```

-17- *Mandates*: […]

-18- *Effects*: Equivalent to:

```
return has_value() ? std::move(val~~*this~~) : static_cast<T>(std::forward<U>(v));
```

8. Modify 22.5.3.8 [optional.monadic] as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let `U` be `invoke_result_t<F, decltype(`*(val)*`~~*val~~)>`.

-2- *Mandates*: […]

-3- *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), val~~*val~~);
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

-4- Let `U` be `invoke_result_t<F, decltype(std::move(`*val*`~~*val~~))>`.

-5- *Mandates*: […]

-6- *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), std::move(val~~*val~~));
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

-7- Let `U` be `remove_cv_t<invoke_result_t<F, decltype(`*(val)*`~~*val~~)>>`.

-8- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), val~~*val~~));
```

is well-formed for some invented variable `u`.

[…]

-9- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), val~~*val~~)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

-10- Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(`*val*`~~*val~~))>>`.

-11- *Mandates*: `U` is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), std::move(val~~*val~~)));
```

is well-formed for some invented variable `u`.

[…]

-12- *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(val~~*val~~))`; otherwise, `optional<U>()`.

9. Modify 22.5.3.9 [optional.mod] as indicated:

```
constexpr void reset() noexcept;
```

-1- *Effects*: If `*this` contains a value, calls ~~`val->`~~*`val.`*`T::~T()` to destroy the contained value; otherwise no effect.

-2- *Postconditions*: `*this` does not contain a value.

10. Modify 22.5.4.2 [optional.ref.ctor] as indicated:

```
template<class U>
  constexpr explicit(!is_convertible_v<U&, T&>)
  optional(optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, U&>);
```

-8- *Constraints*: […]

-9- *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*rhs.operator*());
```

-10- *Remarks*: […]

```
template<class U>
  constexpr explicit(!is_convertible_v<const U&, T&>)
  optional(const optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, const U&>);
```

-11- *Constraints*: […]

-12- *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*rhs.operator*());
```

-13- *Remarks*: […]

```
template<class U>
  constexpr explicit(!is_convertible_v<U, T&>)
  optional(optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, U>);
```

-14- *Constraints*: […]

-15- *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*std::move(rhs).operator*());
```

-16- *Remarks*: […]

```
template<class U>
  constexpr explicit(!is_convertible_v<const U, T&>)
  optional(const optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, const U>);
```

-17- *Constraints*: […]

-18- *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*std::move(rhs).operator*());
```

-19- *Remarks*: […]

4230. `simd<complex>::real/imag` is overconstrained

Section: 29.10.8.4 [simd.complex.access] **Status:** Immediate **Submitter:** Matthias Kretz **Opened:** 2025-03-18 **Last modified:** 2025-11-04

Priority: 2

Discussion:

29.10.8.4 [simd.complex.access] overconstrains the arguments to `real` and `imag`. `complex<T>::real/imag` allows conversions to `T` whereas `simd<complex<T>>` requires basically an exact match (`same_as<simd<T>>` modulo ABI tag differences).

```
complex<double> c = {};
c.real(1.f); // OK

simd<complex<double>> sc = {};
sc.real(simd<float>(1.f)); // ill-formed, should be allowed
```

The design intent was to match the `std::complex<T>` interface. In which case the current wording doesn't match that intent. `complex` doesn't say `real(same_as<T> auto)` but 'real(T)', which allows conversions.

This issue is also present in the `basic_simd(real, imag)` constructor. It deduces the type for the real/imag arguments instead of using a dependent type derived from `value_type` and ABI tag.

```
// OK:
complex<double> c{1., 1.f};

// Ill-formed, should be allowed:
simd<complex<double>> sc0(1., 1.);
simd<complex<double>, 4> sc1(simd<double, 4>(1.), simd<float, 4>(1.f));
```

[2025-06-13; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 29.10.7.1 [simd.overview], class template `basic_simd` synopsis, as indicated:

```
namespace std::datapar {
  template<class T, class Abi> class basic_simd {
  public:
```

```
    using value_type = T;
    using mask_type = basic_simd_mask<sizeof(T), Abi>;
    using abi_type = Abi;

    using real-type = rebind_t<typename T::value_type, basic_simd> // exposition-only

    // 29.10.7.2 [simd.ctor], basic_simd constructors
    […]
    template<simd-floating-point V>
      constexpr explicit(see below) basic_simd(const real-type¥& reals, const real-type¥& imags = {}) noexcept;
    […]
    // 29.10.8.4 [simd.complex.access], basic_simd complex-value accessors
    constexpr real-typeauto real() const noexcept;
    constexpr real-typeauto imag() const noexcept;
    template<simd-floating-point V>
      constexpr void real(const real-type¥& v) noexcept;
    template<simd-floating-point V>
      constexpr void imag(const real-type¥& v) noexcept;
    […]
  };
  […]
}
```

2. Modify 29.10.7.2 [simd.ctor] as indicated:

```
template<simd-floating-point V>
  constexpr explicit(see below) basic_simd(const real-type¥& reals, const real-type¥& imags = {}) noexcept;
```

-19- *Constraints*:

(19.1) — *simd-complex*<basic_simd> is modeled., and

(19.2) — V::size() == size() is true.

[…]

-21- *Remarks*: The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T::value_type` is greater than or equal to the floating-point conversion rank of *real-type*¥`::value_type`.

3. Modify 29.10.8.4 [simd.complex.access] as indicated:

```
constexpr real-typeauto real() const noexcept;
constexpr real-typeauto imag() const noexcept;
```

-1- *Constraints*: *simd-complex*<basic_simd> is modeled.

-2- *Returns*: An object of type *real-type*rebind_t<typename T::value_type, basic_simd> where the i^{th} element is initialized to the result of *cmplx-func*(`operator[](`*i*`)`) for all *i* in the range [`0, size()`), where *cmplx-func* is the corresponding function from <complex>.

```
template<simd-floating-point V>
  constexpr void real(const real-type¥& v) noexcept;
template<simd-floating-point V>
  constexpr void imag(const real-type¥& v) noexcept;
```

-3- *Constraints*:

(3.1) — *simd-complex*<basic_simd> is modeled.,

(3.2) — same_as<typename V::value_type, typename T::value_type> is modeled, and

(3.3) — V::size() == size() is true.

[…]

[2025-07-21; Matthias Kretz comments]

The currently shown P/R says:

> *Remarks*: The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T::value_type` is greater than or equal to the floating-point conversion rank of *real-type*`::value_type`.

But, by construction, *real-type*`::value_type` is the same as `T::value_type`. So we get an elaborately worded `explicit(false)` here (which is correct). Consequently, the proposed resolution needs to strike `explicit(<i>see below</i>)` from 29.10.7.1 [simd.overview] and 29.10.7.2 [simd.ctor] and drop the Remarks paragraph (21).

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 29.10.7.1 [simd.overview], class template `basic_vec` synopsis, as indicated:

```
namespace std::simd {
  template<class T, class Abi> class basic_vec {
    using real-type = see below; // exposition-only
  public:
    using value_type = T;
    using mask_type = basic_mask<sizeof(T), Abi>;
```

```
    using abi_type = Abi;
    using iterator = simd-iterator<basic_vec>;
    using const_iterator = simd-iterator<const basic_vec>;

    // 29.10.7.2 [simd.ctor], basic_vec constructors
    […]
    template<simd-floating-point V>
      constexpr explicit(see below) basic_vec(const real-type¥&& reals, const real-type¥&& imags = {}) noexcept;
    […]
    // 29.10.8.4 [simd.complex.access], basic_vec complex-value accessors
    constexpr real-typeauto real() const noexcept;
    constexpr real-typeauto imag() const noexcept;
    template<simd-floating-point V>
      constexpr void real(const real-type¥& v) noexcept;
    template<simd-floating-point V>
      constexpr void imag(const real-type¥& v) noexcept;
    […]
  };
  […]
}
```

-2- *Recommended practice*: […]

[*Note 1*: …]

-?- If `T` is a specialization of `complex`, *real-type* denotes the same type as `rebind_t<typename T::value_type, basic_vec<T, Abi>>`, otherwise an unspecified non-array object type.

2. Modify 29.10.7.2 [simd.ctor] as indicated:

```
template<simd-floating-point V>
  constexpr explicit(see below)
    basic_vec(const real-type¥&& reals, const real-type¥&& imags = {}) noexcept;
```

-19- *Constraints*:

~~(19.1) —~~ *simd-complex*<basic_vec> is modeled~~,~~ ~~and~~

~~(19.2) — V::size() == size() is true.~~

[…]

~~-21- *Remarks*: The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T::value_type` is greater than or equal to the floating-point conversion rank of `V::value_type`.~~

3. Modify 29.10.8.4 [simd.complex.access] as indicated:

```
constexpr real-typeauto real() const noexcept;
constexpr real-typeauto imag() const noexcept;
```

-1- *Constraints*: *simd-complex*<basic_vec> is modeled.

-2- *Returns*: An object of type *real-type*~~rebind_t<typename T::value_type, basic_vec>~~ where the i^{th} element is initialized to the result of *cmplx-func*(`operator[](`*i*`)`) for all *i* in the range [`0`, `size()`), where *cmplx-func* is the corresponding function from `<complex>`.

```
template<simd-floating-point V>
  constexpr void real(const real-type¥& v) noexcept;
template<simd-floating-point V>
  constexpr void imag(const real-type¥& v) noexcept;
```

-3- *Constraints*:

~~(3.1) —~~ *simd-complex*<basic_vec> is modeled~~,~~

~~(3.2) — same_as<typename V::value_type, typename T::value_type> is modeled, and~~

~~(3.3) — V::size() == size() is true.~~

[…]

4251. Move assignment for `indirect` unnecessarily requires copy construction

Section: 20.4.1.5 [indirect.assign], 20.4.2.5 [polymorphic.assign] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-05-01 **Last modified:** 2025-11-04

Priority: 1

Discussion:

The move assignment operator for `indirect` says:

> *Mandates*: `is_copy_constructible_t<T>` is `true`.

However, the only way it ever construct an object is:

> constructs a new owned object with the owned object of `other` as the argument as an rvalue

and that only ever happens when *alloc* `==` `other.`*alloc* is false.

It seems like we should require `is_move_constructible_v` instead, and only if the allocator traits mean we need to construct an object. (Technically move-constructible might not be correct, because the allocator's `construct` member might use a different constructor).

Additionally, the noexcept-specifier for the move assignment doesn't match the effects. The noexcept-specifier says it can't throw if POCMA is true, but nothing in the effects says that ownership can be transferred in that case; we only do a non-throwing transfer when the allocators are equal. I think we *should* transfer ownership when POCMA is true, which would make the noexcept-specifier correct.

[2025-06-12; Reflector poll]

Set priority to 1 after reflector poll.

Similar change needed for `std::polymorphic`.

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 20.4.1.5 [indirect.assign] as indicated:

```
constexpr indirect& operator=(indirect&& other)
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

-5- *Mandates*: If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `false` and `allocator_traits<Allocator>::is_always_equal::value` is `false`, is ~~copy~~`move_constructible_t<T>` is `true`.

-6- *Effects*: If `addressof(other) == this` is `true`, there are no effects. Otherwise:

(6.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `true`.
(6.2) — If `other` is valueless, `*this` becomes valueless ~~and the owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated~~.
(6.3) — Otherwise, if the allocator needs updating or if *`alloc`* `== other.`*`alloc`* is `true`, ~~swaps the owned objects in `*this` and `other`; the owned object in `other`, if any, is then destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated~~ `*this` takes ownership of the owned object of `other`.
(6.4) — Otherwise, constructs a new owned object with the owned object of `other` as the argument as an rvalue, using either the allocator in `*this` or the allocator in `other` if the allocator needs updating.
(6.5) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
(6.6) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

-7- *Postcondition*: `other` is valueless.

[2025-11-03; Tomasz provides wording.]

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 20.4.1.5 [indirect.assign] as indicated:

```
constexpr indirect& operator=(indirect&& other)
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

-5- *Mandates*: If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `false` and `allocator_traits<Allocator>::is_always_equal::value` is `false`, is ~~copy~~`move_constructible_t<T>` is `true`.

-6- *Effects*: If `addressof(other) == this` is `true`, there are no effects. Otherwise:

(6.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `true`.
(6.2) — If `other` is valueless, `*this` becomes valueless ~~and the owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated~~.
(6.3) — Otherwise, if the allocator needs updating or if *`alloc`* `== other.`*`alloc`* is `true`, ~~swaps the owned objects in `*this` and `other`; the owned object in `other`, if any, is then destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated~~ `*this` takes ownership of the owned object of `other`.
(6.4) — Otherwise, constructs a new owned object with the owned object of `other` as the argument as an rvalue, using ~~either~~ the allocator in `*this` ~~or the allocator in `other` if the allocator needs updating~~.
(6.5) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
(6.6) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

-7- *Postcondition*: `other` is valueless.

2. Modify 20.4.2.5 [polymorphic.assign] as indicated:

```
constexpr polymorphic& operator=(polymorphic&& other)
```

```
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
         allocator_traits<Allocator>::is_always_equal::value);
```

-5- *Mandates*: If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `false` and `allocator_traits<Allocator>::is_always_equal::value` is `false`, `T` is complete type.

-6- *Effects*: If `addressof(other) == this` is `true`, there are no effects. Otherwise:

(6.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is `true`.
(6.?) — If `other` is valueless, `*this` becomes valueless.
(6.2) — Otherwise, if the allocator needs updating or~~If~~ *`alloc`* `==` `other.`*`alloc`* is `true`, ~~swaps the owned objects in `*this` and `other`; the owned object in `other`, if any, is then destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated~~ `*this` takes ownership of the owned object of `other`.
(6.3) — ~~Otherwise, if `alloc != other.alloc` is `true`; if `other` is not valueless, a new owned object is constructed in `*this` using `allocator_traits::construct` with the owned object from~~ Otherwise, constructs a new owned object with the owned object of `other` as the argument as an rvalue, using ~~either~~ the allocator in `*this` ~~or the allocator in `other` if the allocator needs updating~~.
(6.4) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
(6.5) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

[…]

4260. Query objects must be default constructible

Section: 33.2.1 [exec.queryable.general] **Status:** Immediate **Submitter:** Eric Niebler **Opened:** 2025-05-07 **Last modified:** 2025-11-07

Priority: 2

Discussion:

Imported from cplusplus/sender-receiver #333.

We require the types of query objects such as `get_scheduler` to be customization point objects. 16.3.3.3.5 [customization.point.object] requires them to be `semiregular` but that concept does not require default constructability. Much of `std::execution` assumes query object types to be default constructible.

I propose adding a (nothrow) default-constructibility requirement.

[2025-10-23; Reflector poll.]

Set priority to 2 after reflector poll.

"The discussion is wrong, `semiregular` requires `default_initializable`. If we want to mandate nothrow construction (a.k.a the implementation isn't out to get you), I'd rather we do it for all CPOs."

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 33.2.1 [exec.queryable.general] as indicated:

-1- A *queryable object* is a read-only collection of key/value pair where each key is a customization point object known as a *query object*. The type of a query object satisfies `default_initializable`, and its default constructor is not potentially throwing. A *query* is an invocation of a query object with a queryable object as its first argument and a (possibly empty) set of additional arguments. A query imposes syntactic and semantic requirements on its invocations.

[2025-11-05; Tim provides improved wording]

LWG decided to guarantee some additional properties for CPOs.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 16.3.3.3.5 [customization.point.object] as indicated:

-1- A *customization point object* is a function object (22.10 [function.objects]) with a literal class type that interacts with program-defined types while enforcing semantic requirements on that interaction.

-2- The type of a customization point object, ignoring cv-qualifiers, shall model `semiregular` (18.6 [concepts.object]) and shall be a structural type (13.2 [temp.param]) and a trivially copyable type (11.2 [class.prop]). Every constructor of this type shall have a non-throwing exception specification (14.5 [except.spec]).

4272. For `rank == 0`, `layout_stride` is atypically convertible

Section: 23.7.3.4 [mdspan.layout] **Status:** Immediate **Submitter:** Luc Grosheintz **Opened:** 2025-06-02 **Last modified:** 2025-11-05

Priority: 2

View other active issues in [mdspan.layout].

View all other issues in [mdspan.layout].

Discussion:

Commonly, two layouts are considered convertible, if the underlying `extent_types` are convertible.

However, for the two ctors `layout_left::mapping(layout_stride::mapping)` and `layout_right::mapping(layout_stride::mapping)`, the condition is `rank > 0`. Therefore,

```
using E1 = std::extents<int>;
using E2 = std::extents<unsigned int>;

static_assert(std::is_convertible_v<
    std::layout_stride::mapping<E2>,
    std::layout_right::mapping<E1>
  >);
```

even though:

```
static_assert(!std::is_convertible_v<E2, E1>);
```

Moreover, for rank 0 `layout_stride` can be converted to any specialization of `layout_left` or `layout_right`; but not to every specialization of `layout_stride`.

[2025-06-12; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

> [*Drafting note*: As drive-by fixes the edits for `layout_left_padded<>::mapping` and `layout_right_padded<>::mapping` also correct an editorial asymmetry between class header synopsis declaration form and prototype specification form of the corresponding constructors and adjust to the correct formatting of the exposition-only data member *`rank_`*.]

1. Modify 23.7.3.4.5.1 [mdspan.layout.left.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_left::mapping {
    […]
    // 23.7.3.4.5.2 [mdspan.layout.left.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

2. Modify 23.7.3.4.5.2 [mdspan.layout.left.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~extents_type::rank() > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

3. Modify 23.7.3.4.6.1 [mdspan.layout.right.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_right::mapping {
    […]
    // 23.7.3.4.6.2 [mdspan.layout.right.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

4. Modify 23.7.3.4.6.2 [mdspan.layout.right.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(extents_type::rank() > 0see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

5. Modify 23.7.3.4.8.1 [mdspan.layout.leftpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_left_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.8.3 [mdspan.layout.leftpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(extents_type::rank() > 0see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

6. Modify 23.7.3.4.8.3 [mdspan.layout.leftpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(rank_ > 0see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

7. Modify 23.7.3.4.9.1 [mdspan.layout.rightpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_right_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.9.3 [mdspan.layout.rightpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(rank_ > 0see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

8. Modify 23.7.3.4.9.3 [mdspan.layout.rightpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(rank_ > 0see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

[2025-06-20, Luc Grosheintz provides further wording improvements]

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

> [*Drafting note*: As drive-by fixes the edits for `layout_left_padded<>::mapping` and `layout_right_padded<>::mapping` also correct an editorial asymmetry between class header synopsis declaration form and prototype specification form of the corresponding constructors and adjust to the correct formatting of the exposition-only data member *rank_*.]

1. Modify 23.7.3.4.5.1 [mdspan.layout.left.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_left::mapping {
    […]
    // 23.7.3.4.5.2 [mdspan.layout.left.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

2. Modify 23.7.3.4.5.2 [mdspan.layout.left.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~extents_type::rank() > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

3. Modify 23.7.3.4.6.1 [mdspan.layout.right.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_right::mapping {
    […]
    // 23.7.3.4.6.2 [mdspan.layout.right.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

4. Modify 23.7.3.4.6.2 [mdspan.layout.right.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~extents_type::rank() > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

5. Modify 23.7.3.4.8.1 [mdspan.layout.leftpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_left_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.8.3 [mdspan.layout.leftpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

6. Modify 23.7.3.4.8.3 [mdspan.layout.leftpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~rank_ > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

`!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)`

```
template<class LayoutLeftPaddedMapping>
  constexpr explicit(see below)
    mapping(const LayoutLeftPaddedMapping& other);
```

-13- *Constraints*: […]

[…]

-16- *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type> &&
rank_> 1 &&
(padding_value != dynamic_extent ||
 LayoutLeftPaddedMapping::padding_value == dynamic_extent)
```

7. Modify 23.7.3.4.9.1 [mdspan.layout.rightpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_right_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.9.3 [mdspan.layout.rightpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~rank_ > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

8. Modify 23.7.3.4.9.3 [mdspan.layout.rightpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~rank_ > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

`!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)`

```
template<class LayoutRightPaddedMapping>
  constexpr explicit(see below)
    mapping(const LayoutRightPaddedMapping& other);
```

-13- *Constraints*: […]

[…]

-17- *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type> &&
rank_ > 1 &&
(padding_value != dynamic_extent ||
LayoutRightPaddedMapping::padding_value == dynamic_extent)
```

[2025-09-27, Tomasz Kamiński fixes constraints in constructors from padded layouts]

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5008.

[*Drafting note*: As drive-by fixes the edits for `layout_left_padded<>::mapping` and `layout_right_padded<>::mapping` also correct an editorial asymmetry between class header synopsis declaration form and prototype specification form of the corresponding constructors and adjust to the correct formatting of the exposition-only data member *rank_*.]

1. Modify 23.7.3.4.5.1 [mdspan.layout.left.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_left::mapping {
    […]
    // 23.7.3.4.5.2 [mdspan.layout.left.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
```

```
      mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

2. Modify 23.7.3.4.5.2 [mdspan.layout.left.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~extents_type::rank() > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

3. Modify 23.7.3.4.6.1 [mdspan.layout.right.overview] as indicated:

```
namespace std {
  template<class Extents>
  class layout_right::mapping {
    […]
    // 23.7.3.4.6.2 [mdspan.layout.right.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;
    […]
  };
}
```

4. Modify 23.7.3.4.6.2 [mdspan.layout.right.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~extents_type::rank() > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-13- *Constraints*: […]

-14- *Preconditions*: […]

-15- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

5. Modify 23.7.3.4.8.1 [mdspan.layout.leftpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_left_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.8.3 [mdspan.layout.leftpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~extents_type::rank() > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

6. Modify 23.7.3.4.8.3 [mdspan.layout.leftpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~rank_ > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

```
template<class LayoutLeftPaddedMapping>
  constexpr explicit(see below)
    mapping(const LayoutLeftPaddedMapping& other);
```

-13- *Constraints*: […]

[…]

-16- *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type> ||
rank_> 1 &&
(padding_value != dynamic_extent ||
 LayoutLeftPaddedMapping::padding_value == dynamic_extent)
```

7. Modify 23.7.3.4.9.1 [mdspan.layout.rightpad.overview] as indicated:

```
namespace std {
  template<size_t PaddingValue>
  template<class Extents>
  class layout_right_padded<PaddingValue>::mapping {
    […]
    // 23.7.3.4.9.3 [mdspan.layout.rightpad.cons], constructors
    […]
    template<class OtherExtents>
      constexpr explicit(~~rank_ > 0~~see below)
        mapping(const layout_stride::mapping<OtherExtents>&);
    […]
  };
}
```

8. Modify 23.7.3.4.9.3 [mdspan.layout.rightpad.cons] as indicated:

```
template<class OtherExtents>
  constexpr explicit(~~rank_ > 0~~see below)
    mapping(const layout_stride::mapping<OtherExtents>& other);
```

-10- *Constraints*: […]

-11- *Preconditions*: […]

-12- *Effects*: […]

-?- *Remarks*: The expression inside `explicit` is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

```
template<class LayoutRightPaddedMapping>
  constexpr explicit(see below)
    mapping(const LayoutRightPaddedMapping& other);
```

-13- *Constraints*: […]

[…]

-17- *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type> ||
rank_ > 1 &&
(padding_value != dynamic_extent ||
LayoutRightPaddedMapping::padding_value == dynamic_extent)
```

4302. Problematic `vector_sum_of_squares` wording

Section: 29.9.13.8 [linalg.algs.blas1.ssq] **Status:** Immediate **Submitter:** Mark Hoemmen **Opened:** 2025-07-23 **Last modified:** 2025-11-05

Priority: 1

Discussion:

Addresses US 169-276

The current wording for `vector_sum_of_squares` 29.9.13.8 [linalg.algs.blas1.ssq] has three problems with its specification of the value of `result.scaling_factor`.

1. The function permits `InVec::value_type` and `Scalar` to be any linear algebra value types. However, computing `result.scaling_factor` that satisfies both (3.1) and (3.2) requires more operations, such as division. Even if those operations are defined, they might not make `result.scaling_factor` satisfy the required properties. For example, integers have division, but integer division won't help here.

2. LAPACK's xLASSQ (the algorithm to which Note 1 in 29.9.13.8 [linalg.algs.blas1.ssq] refers) changed its algorithm recently (see Reference-LAPACK/lapack/pull/#494) so that the scaling factor is no longer necessarily the maximum of the input scaling factor and the absolute value of all the input elements. It's a better algorithm and we would like to be able to use it.

3. Both members of `sum_of_squares_result<Scalar>` have the same type, `Scalar`. If the input `mdspan`'s `value_type` represents a quantity with units, this would not be correct. For example, if `value_type` has units of distance (say `[m]`), the sum of squares should have units of area (`[m`^{2}`]`), while the scaling factor should have units of distance (`[m]`).

Problem (1) means that the current wording is broken. I suggest two different ways to fix this.

1. Remove `vector_sum_of_squares` entirely (both overloads from 29.9.2 [linalg.syn], and the entire 29.9.13.8 [linalg.algs.blas1.ssq]). That way, we won't be baking an old, less good algorithm into the Standard. Remove Note 3 from 29.9.13.9 [linalg.algs.blas1.nrm2], which is the only other reference to `vector_sum_of_squares` in the Standard.

2. Fix 29.9.13.8 [linalg.algs.blas1.ssq] by adding to the *Mandates* element (para 2) that `InVec::value_type` and `Scalar` are both floating-point types (so that we could fix this later if we want), and remove 29.9.13.8 [linalg.algs.blas1.ssq] 3.1. Optionally add *Recommended Practice*, though Note 1 already suggests the intent.

I prefer just removing `vector_sum_of_squares`. Implementers who care about QoI of `vector_two_norm` should already know what to do. If somebody cares sufficiently, they can propose it back for C++29 and think about how to make it work for generic number types.

[2025-10-17; Reflector poll. Status changed: New → LEWG.]

Set priority to 1 after reflector poll. Send to LEWG.

This is the subject of NB comment 169-276. LWG took a poll in the 2025-10-10 telecon and recommends that LEWG confirms this resolution.

[Kona 2025-11-05; approved by LEWG to resolve US 169-276.]

[Kona 2025-11-05; approved by LWG. Status changed: LEWG → Immediate.]

Proposed resolution:

This wording is relative to N5014.

> [*Drafting note:* The wording below implements option 1 of the issue discussion]

1. Modify 29.9.2 [linalg.syn], header `<linalg>` synopsis, as indicated:

```
namespace std::linalg {
  […]
  // 29.9.13.8 [linalg.algs.blas1.ssq], scaled sum of squares of a vector's elements
  template<class Scalar>
    struct sum_of_squares_result {
      Scalar scaling_factor;
    };
  template<in-vector InVec, class Scalar>
    sum_of_squares_result<Scalar>
      vector_sum_of_squares(InVec v, sum_of_squares_result<Scalar> init);
  template<class ExecutionPolicy, in-vector InVec, class Scalar>
    sum_of_squares_result<Scalar>
      vector_sum_of_squares(ExecutionPolicy&& exec,
                            InVec v, sum_of_squares_result<Scalar> init);
  […]
}
```

2. Delete the entire 29.9.13.8 [linalg.algs.blas1.ssq] as indicated:

29.9.13.8 Scaled sum of squares of a vector's elements [linalg.algs.blas1.ssq]

```
template<in-vector InVec, class Scalar>
  sum_of_squares_result<Scalar> vector_sum_of_squares(InVec v, sum_of_squares_result<Scalar> init);
template<class ExecutionPolicy, in-vector InVec, class Scalar>
  sum_of_squares_result<Scalar> vector_sum_of_squares(ExecutionPolicy&& exec,
                                                      InVec v, sum_of_squares_result<Scalar> init);
```

-1- [*Note 1*: These functions correspond to the LAPACK function xLASSQ[20]. — *end note*]

-2- *Mandates*: `decltype(abs-if-needed(declval<typename InVec::value_type>()))` is convertible to `Scalar`.

-3- *Effects*: Returns a value `result` such that

(3.1) — `result.scaling_factor` is the maximum of `init.scaling_factor` and `abs-if-needed(x[i])` for all `i` in the domain of `v`; and

(3.2) — let `s2init` be

`init.scaling_factor * init.scaling_factor * init.scaled_sum_of_squares`

then `result.scaling_factor * result.scaling_factor * result.scaled_sum_of_squares` equals the sum of `s2init` and the squares of `abs-if-needed(x[i])` for all `i` in the domain of `v`.

-4- *Remarks*: If `InVec::value_type`, and `Scalar` are all floating-point types or specializations of `complex`, and if `Scalar` has higher precision than `InVec::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

3. Modify 29.9.13.9 [linalg.algs.blas1.nrm2] as indicated:

```
template<in-vector InVec, class Scalar>
  Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, class Scalar>
  Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
```

-1- [*Note 1*: […]]

-2- *Mandates*: […]

-3- *Returns*: […]

[*Note 2*: […]]

-4- *Remarks*: […]

~~[*Note 3*: An implementation of this function for floating-point types `T` can use the `scaled_sum_of_squares` result from `vector_sum_of_squares(x, {.scaling_factor=1.0, .scaled_sum_of_squares=init})`. — *end note*]~~

4304. `std::optional<NonReturnable&>` is ill-formed due to `value_or`

Section: 22.5.4.6 [optional.ref.observe] **Status:** Immediate **Submitter:** Jiang An **Opened:** 2025-07-25 **Last modified:** 2025-11-04

Priority: 1

Discussion:

Currently, if `T` is an array type or a function type, instantiation of `std::optional<T&>` is still ill-formed, because the return type of its `value_or` member function is specified as `remove_cv_t<T>`, which is invalid as a return type.

However, we don't exclude such `T&` from valid contained types. Given only `value_or` is problematic here, perhaps we can avoid providing it if `T` is not returnable.

[2025-10-16; Reflector poll]

Set priority to 1 after reflector poll.

Why not just add *Constraints*: and use `decay_t<T>` for the return type, instead of "not always present" which is currently only used for member types, not member functions.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 22.5.4.1 [optional.optional.ref.general], header `<iterator>` synopsis, as indicated:

```
namespace std {
  template<class T>
  class optional<T&> {
    […]
    constexpr T& value() const; // freestanding-deleted
    template<class U = remove_cv_t<T>>
      constexpr remove_cv_t<T> value_or(U&& u) const; // not always present
    […]
  };
}
```

2. Modify 22.5.4.6 [optional.ref.observe] as indicated:

```
template<class U = remove_cv_t<T>> constexpr remove_cv_t<T> value_or(U&& u) const;
```

-8- Let `X` be `remove_cv_t<T>`.

-9- *Mandates*: `is_constructible_v<X, T&> && is_convertible_v<U, X>` is `true`.

-10- *Effects*: Equivalent to:

```
return has_value() ? *val : static_cast<X>(std::forward<U>(u));
```

-?- *Remarks*: This function template is present if and only if `T` is a non-array object type.

[2025-10-16; Jonathan provides new wording]

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 22.5.4.6 [optional.ref.observe] as indicated:

```
template<class U = remove_cv_t<T>> constexpr remove_cv_t<T> value_or(U&& u) const;
```

-?- *Constraints*: `T` is a non-array object type.

-8- Let `X` be `remove_cv_t<T>`.

-9- *Mandates*: `is_constructible_v<X, T&> && is_convertible_v<U, X>` is `true`.

-10- *Effects*: Equivalent to:

```
return has_value() ? *val : static_cast<X>(std::forward<U>(u));
```

-?- *Remarks*: The return type is unspecified if `T` is an array type or a non-object type. [*Note ?*: This is to avoid the declaration being ill-formed. — *end note*]

4308. `std::optional<T&>::iterator` can't be a contiguous iterator for some `T`

Section: 22.5.4.5 [optional.ref.iterators] **Status:** Immediate **Submitter:** Jiang An **Opened:** 2025-08-05 **Last modified:** 2025-11-06

Priority: 1

Discussion:

This is related to LWG 4304[(i)]. When `T` is function type or an incomplete array type, it is impossible to implement all requirements in 22.5.4.5 [optional.ref.iterators]/1.

When `T` is an incomplete object type, we may want to support `std::optional<T&>` as it's sometimes a replacement of `T*`. Perhaps we can require that the iterator type is always a random access iterator, and additional models `contiguous_iterator` when `T` is complete.

When `T` is a function type, the possibly intended iterator would be not even an actual iterator. But it seems that range-for loop over such an `std::optional<T&>` can work.

[2025-08-29; Reflector poll]

Set priority to 1 after reflector poll.

"How can `end()` work for a pointer to incomplete type? `begin`/`end` should be constrained on object types, and Mandate complete object types. The aliases shouldn't be defined for non-object types, but probably harmless."

[Kona 2025-11-05; Should only be a range for an object type.]

`optional<T&>` doesn't currently allow incomplete types anyway.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 22.5.4.5 [optional.ref.iterators] as indicated:

```
using iterator = implementation-defined;
```

-1- ~~T~~If `T` is an object type, this type models ~~`contiguous_iterator` (24.3.4.14 [iterator.concept.contiguous])~~`random_access_iterator` (24.3.4.13 [iterator.concept.random.access]), meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7 [random.access.iterators]), and meets the requirements for constexpr iterators (24.3.1 [iterator.requirements.general]), with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator`. When `T` is a complete object type, `iterator` additionally models `contiguous_iterator` (24.3.4.14 [iterator.concept.contiguous]).

~~-2-~~ All requirements on container iterators (23.2.2.2 [container.reqmts]) apply to `optional::iterator`.

-?- If `T` is a function type, `iterator` supports all operators required by the `random_access_iterator` concept (24.3.4.13 [iterator.concept.random.access]) along with the `<=>` operator as specified for container iterators (23.2.2.2 [container.reqmts]). `iterator` dereferences to a `T` lvalue. These operators behave as if `iterator` were an actual iterator iterating over a range of `T`, and result in constant subexpressions whenever the behavior is well-defined. [*Note ?*: Such an `optional::iterator` does not need to declare any member type because it is not an actual iterator type. — *end note*]

[Kona 2025-11-06, Tomasz provides updated wording]

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

1. Modify 22.5.4.1 [optional.optional.ref.general] as indicated:

```
namespace std {
  template<class T>
  class optional<T&> {
  public:
    using value_type     = T;
    using iterator       = implementation-defined;  // present only if T is an object type other than an array of unknown bound; se
  public:
    […]

    // [optional.ref.iterators], iterator support
    constexpr ~~iterator~~auto begin() const noexcept;
    constexpr ~~iterator~~auto end() const noexcept;

    […]
  };
}
```

2. Modify 22.5.4.5 [optional.ref.iterators] as indicated:

```
using iterator = implementation-defined; // present only if T is an object type other than an array of unknown bound
```

-1- This type models `contiguous_iterator` (24.3.4.14 [iterator.concept.contiguous]), meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7 [random.access.iterators]), and meets the requirements for constexpr iterators (24.3.1 [iterator.requirements.general]), with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator`.

-2- All requirements on container iterators (23.2.2.2 [container.reqmts]) apply to `optional::iterator`.

```
constexpr ~~iterator~~auto begin() const noexcept;
```

-?- *Constraints*: `T` is an object type other than an array of unknown bound.

-3- Returns: An object `i` of type `iterator`, such that ~~If `has_value()` is `true`,~~ `i` is an iterator referring to `*val` if `has_value()` is `true`, and ~~Otherwise,~~ a past-the-end iterator value otherwise.

```
constexpr ~~iterator~~auto end() const noexcept;
```

-?- *Constraints*: T is an object type other than an array of unknown bound.

-4- Returns: `begin() + has_value()`.

4316. `{can_}substitute` specification is ill-formed

Section: 21.4.13 [meta.reflection.substitute] **Status:** Immediate **Submitter:** Matthias Wippich **Opened:** 2025-08-15 **Last modified:** 2025-11-05

Priority: 1

Discussion:

Addresses US 114-175

`can_substitute` and `substitute` are currently specified in terms of splices in a template argument list:

21.4.13 [meta.reflection.substitute] p3:

> *Returns*: `true` if `Z<[:Args:]...>` is a valid *template-id* (13.3 [temp.names]) that does not name a function whose type contains an undeduced placeholder type. Otherwise, `false`.

21.4.13 [meta.reflection.substitute] p7:

> *Returns*: `^^Z<[:Args:]...>`.

This wording was introduced in P2996R11. However, merging in changes from P3687 "Final Adjustments to C++26 Reflection" in P2996R13 changed the rules for splices in this context. This makes `can_substitute` and `substitute` as specified currently ill-formed. We cannot use the given syntax to splice an arbitrary choice of values, types and templates anymore.

While the intent seems clear, this should be rephrased to be more technically correct.

[2025-10-22; Reflector poll.]

Set priority to 1 after reflector poll.

[2025-10-27; Tomasz provides wording.]

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 21.4.13 [meta.reflection.substitute] as indicated:

-1- For value `x` of type `info`, and prvalue constant expression `X` that computes the reflection held by `x`, let *`TARG-SPLICE`*`(x)` be:

- -1.1- `template [: X :]` if `is_template(x)` is `true`, otherwise
- -1.2- `typename [: X :]` if `is_type(x)` is `true`, otherwise
- -1.3- `([: X :])`

```
template<reflection_range R = initializer_list<info>>
  consteval bool can_substitute(info templ, R&& arguments);
```

-1- ~~Let `Z` be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.~~ Let n be the number of elements in `arguments`, and e_i be the i^{th} element of `arguments`.

-2- *Returns*: `true` if `Z<~~[:Args:]...~~`*`TARG-SPLICE`*(e_0), ..., *`TARG-SPLICE`*(e_{n-1})`>` is a valid *template-id* (13.3 [temp.names]) that does not name a function whose type contains an undeduced placeholder type. Otherwise, `false`.

-3- *Throws*: `meta::exception` unless `templ` represents a template, and every reflection in `arguments` represents a construct usable as a template argument (13.4 [temp.arg]).

-4- [*Note*: If forming `Z<~~[:Args:]...~~`*`TARG-SPLICE`*(e_0), ..., *`TARG-SPLICE`*(e_{n-1})`>` leads to a failure outside of the immediate context, the program is ill-formed. — *end note*]

```
template<reflection_range R = initializer_list<info>>
  consteval info substitute(info templ, R&& arguments);
```

-5- ~~Let `Z` be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.~~ Let n be the number of elements in `arguments`, and e_i be the i^{th} element of `arguments`.

-6- *Returns*: `^^Z<~~[:Args:]...~~`*`TARG-SPLICE`*(e_0) ..., *`TARG-SPLICE`*(e_{n-1})`>`.

-7- *Throws*: `meta::exception` unless `can_substitute(templ, arguments)` is `true`.

-8- [*Note*: If forming `Z<~~[:Args:]...~~`*`TARG-SPLICE`*(e_0), ..., *`TARG-SPLICE`*(e_{n-1})`>` leads to a failure outside of the immediate context, the program is ill-formed. — *end note*]

[2025-10-27; Reflector comments.]

We lost definition of `Z`. Use *`TARG-SPLICE`*`([:Args:])...`.

[2025-11-03; Tomasz provides wording.]

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.13 [meta.reflection.substitute] as indicated:

> -1- Let *`TARG-SPLICE`*`(x)` be:
>
> - -1.1- `template [: x :]` if `is_template(x)` is `true`, otherwise
> - -1.2- `typename [: x :]` if `is_type(x)` is `true`, otherwise
> - -1.3- `([: x :])`

```
template<reflection_range R = initializer_list<info>>
  consteval bool can_substitute(info templ, R&& arguments);
```

> -1- Let `Z` be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.
>
> -2- *Returns*: `true` if `Z<`*`TARG-SPLICE`*`(~~[:~~Args~~:]~~)`**`...`**`>` is a valid *template-id* (13.3 [temp.names]) that does not name a function whose type contains an undeduced placeholder type. Otherwise, `false`.
>
> -3- *Throws*: `meta::exception` unless `templ` represents a template, and every reflection in `arguments` represents a construct usable as a template argument (13.4 [temp.arg]).
>
> -4- [*Note*: If forming `Z<`*`TARG-SPLICE`*`(~~[:~~Args~~:]~~)`**`...`**`>` leads to a failure outside of the immediate context, the program is ill-formed. — *end note*]

```
template<reflection_range R = initializer_list<info>>
  consteval info substitute(info templ, R&& arguments);
```

> -5- Let `Z` be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.
>
> -6- *Returns*: `Z<`*`TARG-SPLICE`*`(~~[:~~Args~~:]~~)`**`...`**`>`.
>
> -7- *Throws*: `meta::exception` unless `can_substitute(templ, arguments)` is `true`.
>
> -8- [*Note*: If forming `Z<`*`TARG-SPLICE`*`(~~[:~~Args~~:]~~)`**`...`**`>` leads to a failure outside of the immediate context, the program is ill-formed. — *end note*]

4358. §[exec.as.awaitable] is using "Preconditions:" when it should probably be described in the constraint

Section: 33.13.1 [exec.as.awaitable] **Status:** Immediate **Submitter:** Lewis Baker **Opened:** 2025-08-27 **Last modified:** 2025-11-06

Priority: 2

View other active issues in [exec.as.awaitable].

View all other issues in [exec.as.awaitable].

Discussion:

In 33.13.1 [exec.as.awaitable] bullet 7.2 it states:

> (7.2) — Otherwise, `(void(p), expr)` if *`is-awaitable`*`<Expr, U>` is `true`, where `U` is an unspecified class type that is not `Promise` and that lacks a member named `await_transform`.
>
> *Preconditions*: *`is-awaitable`*`<Expr, Promise>` is `true` and the expression `co_await expr` in a coroutine with promise type `U` is expression-equivalent to the same expression in a coroutine with promise type `Promise`.

The "*Preconditions*:" sentence there refers to static properties of the program and so seems like a better fit for a *Mandates*: element or for folding into the constraint.

Also, in the part of the precondition above which says "… and the expression `co_await expr` in a coroutine with promise type `U` is expression-equivalent to the same expression in a coroutine with promise type `Promise`" it is unclear how this can be satisfied, as the types involved are different and therefore the expression cannot be expression-equivalent.

I think perhaps what is intended here is something along the lines of the first expression having "effects equivalent to" the second expression, instead of "expression-equivalent to"?

However, I think there is a more direct way to express the intent here, by instead just requiring that `decltype(`*`GET-AWAITER`*`(expr))` satisfies *`is-awaiter`*`<Promise>`. This checks whether `expr` would be a valid type to return from a `Promise::await_transform()` function.

[2025-10-23; Reflector poll.]

Set priority to 2 after reflector poll.

"Intent of the original wording seems to be that GET-AWAITER(expr) should be the same as GET-AWAITER(expr, p) and this rewording loses that. Don't understand the rationale for the new wording either." (More details in the reflector thread in Sept. 2025)

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.13.1 [exec.as.awaitable] as indicated:

> -7- `as_awaitable` is a customization point object. For subexpressions `expr` and `p` where `p` is an lvalue, `Expr` names the type `decltype((expr))` and `Promise` names the type `decay_t<decltype((p))>`, `as_awaitable(expr, p)` is expression-equivalent to, except that the evaluations of `expr` and `p` are indeterminately sequenced:
>
> (7.1) — `expr.as_awaitable(p)` if that expression is well-formed.
>
> *Mandates*: *`is-awaitable`*`<A, Promise>` is `true`, where `A` is the type of the expression above.
>
> (7.2) — Otherwise, `(void(p), expr)` if `decltype(`*`GET-AWAITER`*`(expr))` satisfies *`is-awaiter`*`<Promise>`~~*`.is-awaitable`*`<Expr, U>` is `true`, where `U` is an unspecified class type that is not `Promise` and that lacks a member named `await_transform`.~~
>
> ~~*Preconditions*: *`is-awaitable`*`<Expr, Promise>` is `true` and the expression `co_await expr` in a coroutine with promise type `U` is expression-equivalent to the same expression in a coroutine with promise type `Promise`.~~
>
> (7.3) — […]
>
> (7.4) — […]
>
> (7.5) — […]

4360. *`awaitable-sender`* concept should qualify use of *`awaitable-receiver`* type

Section: 33.13.1 [exec.as.awaitable] **Status:** Immediate **Submitter:** Lewis Baker **Opened:** 2025-08-27 **Last modified:** 2025-11-06

Priority: 2

View other active issues in [exec.as.awaitable].

View all other issues in [exec.as.awaitable].

Discussion:

In 33.13.1 [exec.as.awaitable] p1 there is an exposition-only helper concept *`awaitable-sender`* defined as follows:

```
namespace std::execution {
  template<class Sndr, class Promise>
    concept awaitable-sender =
      single-sender<Sndr, env_of_t<Promise>> &&
      sender_to<Sndr, awaitable-receiver> && // see below
      requires (Promise& p) {
        { p.unhandled_stopped() } -> convertible_to<coroutine_handle<>>;
      };
}
```

The mention of the type *`awaitable-receiver`* here does not refer to any exposition-only type defined at namespace-scope. It seems to, instead, be referring to the nested member-type *`sender-awaitable`*`<Sndr, Promise>::`*`awaitable-receiver`* and so should be qualified as such.

[2025-10-23; Reflector poll.]

Set priority to 2 after reflector poll.

"We should move the declaration of *`sender-awaitable`* before the concept."

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.13.1 [exec.as.awaitable] as indicated:

> -1- `as_awaitable` transforms an object into one that is awaitable within a particular coroutine. Subclause 33.13 [exec.coro.util] makes use of the following exposition-only entities:
>
> ```
> namespace std::execution {
>   template<class Sndr, class Promise>
>     concept awaitable-sender =
>       single-sender<Sndr, env_of_t<Promise>> &&
>       sender_to<Sndr, typename sender-awaitable<Sndr, Promise>::awaitable-receiver> && // see below
>       requires (Promise& p) {
>         { p.unhandled_stopped() } -> convertible_to<coroutine_handle<>>;
>       };
>   […]
> }
> ```

4369. *check-types* function for upon_error and upon_stopped is wrong

Section: 33.9.12.9 [exec.then] **Status:** Immediate **Submitter:** Eric Niebler **Opened:** 2025-08-31 **Last modified:** 2025-11-06

Priority: 2

Discussion:

Addresses US 219-350

The following has been reported by Trevor Gray:

In 33.9.12.9 [exec.then] p5, the *impls-for<decayed-typeof<then-cpo>>::check-types* unction is specified as follows:

> ```
> template<class Sndr, class... Env>
>   static consteval void check-types();
> ```
>
> *Effects*: Equivalent to:
>
> ```
> auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
> auto fn = []<class... Ts>(set_value_t(*)(Ts...)) {
>   if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>, Ts...>)
>     throw unspecified-exception();
> };
> cs.for-each(overload-set{fn, [](auto){}});
> ```
>
> where *unspecified-exception* is a type derived from exception.

The line `auto fn = []<class... Ts>(set_value_t(*)(Ts...)) {` is correct when *then-cpo* is then but not when it is upon_error or upon_stopped.

For upon_error it should be:

```
auto fn = []<class... Ts>(set_error_t(*)(Ts...)) {
```

and for upon_stopped it should be:

```
auto fn = []<class... Ts>(set_stopped_t(*)(Ts...)) {
```

We can achieve that by replacing set_value_t in the problematic line with *decayed-typeof<set-cpo>*.

[2025-10-23; Reflector poll.]

Set priority to 2 after reflector poll.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.12.9 [exec.then] as indicated:

```
template<class Sndr, class... Env>
  static consteval void check-types();
```

-5- *Effects:* Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
auto fn = []<class... Ts>(~~set_value_t~~decayed-typeof<set-cpo>(*)(Ts...)) {
  if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>, Ts...>)
    throw unspecified-exception();
};
cs.for-each(overload-set{fn, [](auto){}});
```

where *unspecified-exception* is a type derived from exception.

4376. ABI tag in return type of [simd.mask.unary] is overconstrained

Section: 29.10.9.4 [simd.mask.unary] **Status:** Immediate **Submitter:** Matthias Kretz **Opened:** 2025-09-15 **Last modified:** 2025-11-04

Priority: 1

View other active issues in [simd.mask.unary].

View all other issues in [simd.mask.unary].

Discussion:

Addresses DE 298

29.10.9.4 [simd.mask.unary] spells out the return type with the ABI tag of the basic_mask specialization. That's problematic / overconstrained.

1. Consider Intel SandyBridge/IvyBridge-like targets:

```
vec<float>::size() -> 8
vec<int>::size() -> 4
mask<float>::size() -> 8
```

The ABI tag in this case encodes for `vec<float>` that one object holds 8 elements and is passed via *one* register. `vec<int>` uses a different ABI tag that says 4 elements passed via *one* register. `vec<int, 8>`'s ABI tag says 8 elements passed via *two* registers.

Now what should `+mask<float>()` return? The working draft says it must return a `basic_vec<int, mask<float>::abi_type>`. And `mask<float>::abi_type` is constrained to be the same as `vec<float>::abi_type`. The working draft thus makes it impossible to implement ABI tags that encode number of elements + number of registers (+ bit-masks vs. vector-masks, but that's irrelevant for this issue). Instead, an ABI tag would have to encode the native SIMD width of all vectorizable types. And that's unnecessarily making compatible types incompatible. Also we make it harder to add to the set of vectorizable types in the future.

2. The issue is even worse for an implementation that implements `vec<complex<T>>` using different ABI tags. Encoding whether the value-type is complex into the ABI is useful because it impacts how the mask is stored (`mask<complex<float>, 8>` is internally stored as a 16-element bit-mask (for interleaved `complex`), while `mask<double, 8>` is stored as an 8-element bit-mask). The ABI tag can also be used to implement interleaved vs. contiguous storage, which is useful for different architectures. If we require `+mask<complex<float>>()` to be of a different type than any `vec<long long>` would ever be, that's just brittle and unnecessary template bloat.

[2025-10-17; Reflector poll.]

Set priority to 1 after reflector poll.

"Should be addressed together with 4238(i)."

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

[*Drafting note:* LWG 4238(i) is closely related.]

1. Modify 29.10.2 [simd.expos] as indicated:

```
using simd-size-type = see below;                       // exposition only
template<size_t Bytes> using integer-from = see below; // exposition only

template<class T, class Abi>
  constexpr simd-size-type simd-size-v = see below;              // exposition only
template<class T> constexpr size_t mask-element-size = see below; // exposition only

template <size_t Bytes, class Abi>
  using simd-vec-from-mask-t = see below;                       // exposition only
[…]
```

2. Modify 29.10.2.1 [simd.expos.defn] as indicated:

```
template<class T> constexpr size_t mask-element-size = see below; // exposition only
```

-4- *mask-element-size*<basic_mask<Bytes, Abi>> has the value `Bytes`.

```
template <size_t Bytes, class Abi>
  using simd-vec-from-mask-t = see below;
```

-?- *simd-vec-from-mask-t*<Bytes, Abi> is an alias for an enabled specialization of `basic_vec` if and only if `basic_mask<Bytes, Abi>` is a data-parallel type and *integer-from*<Bytes> is valid and a vectorizable type.

-?- *simd-vec-from-mask-t*<Bytes, Abi>`::size() == basic_mask<Bytes, Abi>::size()` is `true`.

-?- `typename` *simd-vec-from-mask-t*<Bytes, Abi>`::value_type` is *integer-from*<Bytes>

3. Modify 29.10.9.1 [simd.mask.overview], class template `basic_mask` overview synopsis, as indicated:

```
namespace std::simd {
  template<size_t Bytes, class Abi> class basic_mask {
  public:
    […]
    // 29.10.9.4 [simd.mask.unary], basic_mask unary operators
    constexpr basic_mask operator!() const noexcept;
    constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator+() const noexcept;
    constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator-() const noexcept;
    constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator~() const noexcept;
    […]
}
```

4. Modify 29.10.9.4 [simd.mask.unary] as indicated:

```
constexpr basic_mask operator!() const noexcept;
constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator+() const noexcept;
constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator-() const noexcept;
constexpr ~~basic_vec<integer-from<Bytes>, Abi>~~simd-vec-from-mask-t<Bytes, Abi> operator~() const noexcept;
```

-1- Let *op* be the operator.

-2- *Returns*: […]

[2025-11-04; Matthias Kretz provides new wording]

This also resolves 4238(i) and addresses DE 297.

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 29.10.9.1 [simd.mask.overview], class template `basic_mask overview` synopsis, as indicated:

```
namespace std::simd {
  template<size_t Bytes, class Abi> class basic_mask {
  public:
    […]
    // 29.10.9.4 [simd.mask.unary], basic_mask unary operators
    constexpr basic_mask operator!() const noexcept;
    constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator+() const noexcept;
    constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator-() const noexcept;
    constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator~() const noexcept;
    […]
}
```

2. Modify 29.10.9.4 [simd.mask.unary] as indicated:

```
constexpr basic_mask operator!() const noexcept;
constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator+() const noexcept;
constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator-() const noexcept;
constexpr ~~basic_vec<integer_from<Bytes>, Abi>~~see below operator~() const noexcept;
```

-1- Let *op* be the operator.

-2- *Returns*: A data-parallel object where the *i*-th element is initialized to the results of applying *op* to `operator[](i)` for all *i* in the range of [0, `size()`).

-?- *Remarks*: If there exists a vectorizable signed integer type `I` such that `sizeof(I) == Bytes` is `true`, `operator+`, `operator-`, and `operator~` return an enabled specialization `R` of `basic_vec` such that `R::value_type` denotes *integer-from*<Bytes> and `R::size() == size()` is `true`. Otherwise, these operators are defined as deleted and their return types are unspecified.

4383. `constant_wrapper`'s pseudo-mutators are underconstrained

Section: 21.3.5 [const.wrap.class] **Status:** New **Submitter:** Hewill Kang **Opened:** 2025-09-24 **Last modified:** 2025-11-07

Priority: 1

Discussion:

Unlike other operators, `constant_wrapper`'s pseudo-mutators only require that the wrapped type has corresponding mutators, but do not require them to be `constexpr` or to return a sensible value. This inconsistency loses the SFINAE friendliness (demo):

```
#include <type_traits>

void test(auto t) {
  if constexpr (requires { +t; })  // ok
    +t;
  if constexpr (requires { -t; })  // ok
    -t;
  if constexpr (requires { ++t; }) // hard error
    ++t;
  if constexpr (requires { --t; }) // hard error
    --t;
}

struct S {
  /* constexpr */ int operator+() const { return 0; }
  /* constexpr */ int operator++() { return 0; }
  constexpr void operator-() const { }
  constexpr void operator--() { }
};

int main() {
  test(std::cw<S{}>);
}
```

Since these pseudo-mutators have constraints, it is reasonable to further require constant expressions.

[2025-10-17; Reflector poll.]

Set priority to 1 after reflector poll.

`operator+=` changed between P2781R4 and P2781R5, intent is unclear.

Previous resolution [SUPERSEDED]:

> This wording is relative to N5014.
>
> 1. Modify 21.3.5 [const.wrap.class], class template `constant_wrapper` synopsis, as indicated:
>
> [*Drafting note:* The requires clause follows the form of `constant_wrapper`'s function call operator.]

```
struct cw-operators {                                                    // exposition only
  […]
  // pseudo-mutators
  template<constexpr-param T>
    constexpr auto operator++(this T) noexcept
      requires requires(T::value_type x) { constant_wrapper<++x>(); }
        { return constant_wrapper<[] { auto c = T::value; return ++c; }()>{}; }
  template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept
      requires requires(T::value_type x) { constant_wrapper<x++>(); }
        { return constant_wrapper<[] { auto c = T::value; return c++; }()>{}; }

  template<constexpr-param T>
    constexpr auto operator--(this T) noexcept
      requires requires(T::value_type x) { constant_wrapper<--x>(); }
        { return constant_wrapper<[] { auto c = T::value; return --c; }()>{}; }
  template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept
      requires requires(T::value_type x) { constant_wrapper<x-->(); }
        { return constant_wrapper<[] { auto c = T::value; return c--; }()>{}; }

  template<constexpr-param T, constexpr-param R>
    constexpr auto operator+=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x += R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v += R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator-=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x -= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v -= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator*=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x *= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v *= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator/=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x /= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v /= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator%=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x %= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v %= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator&=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x &= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v &= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator|=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x |= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v |= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator^=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x ^= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v ^= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator<<=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x <<= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v <<= R::value; }()>{}; }
  template<constexpr-param T, constexpr-param R>
    constexpr auto operator>>=(this T, R) noexcept
      requires requires(T::value_type x) { constant_wrapper<x >>= R::value>(); }
        { return constant_wrapper<[] { auto v = T::value; return v >>= R::value; }()>{}; }
};
```

[2025-11-05; Zach provides improved wording]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.3.5 [const.wrap.class], class template `constant_wrapper` synopsis, as indicated:

> [*Drafting note:* The requires clause follows the form of `constant_wrapper`'s function call operator.]

```
struct cw-operators {                                                    // exposition only
  […]
  // pseudo-mutators
  template<constexpr-param T>
    constexpr auto operator++(this T) noexcept requires requires(T::value_type x) { ++x; }
      { return constant_wrapper<[] { auto c = T::value; return ++c; }()>{}; }
  template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept requires requires(T::value_type x) { x++; }
      { return constant_wrapper<[] { auto c = T::value; return c++; }()>{}; }

  template<constexpr-param T>
    constexpr auto operator--(this T) noexcept requires requires(T::value_type x) { --x; }
      { return constant_wrapper<[] { auto c = T::value; return --c; }()>{}; }
  template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept requires requires(T::value_type x) { x--; }
      { return constant_wrapper<[] { auto c = T::value; return c--; }()>{}; }

  template<constexpr-param T, constexpr-param R>
    constexpr auto operator+=(this T, R) noexcept requires requires(T::value_type x) { x += R::value; }
      { return constant_wrapper<[] { auto v = T::value; return v += R::value; }()>{}; }
```

```
    template<constexpr param T, constexpr param R>
      constexpr auto operator -=(this T, R) noexcept requires requires(T::value_type x) { x -= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v -= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator*=(this T, R) noexcept requires requires(T::value_type x) { x *= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v *= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator/=(this T, R) noexcept requires requires(T::value_type x) { x /= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v /= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator%=(this T, R) noexcept requires requires(T::value_type x) { x %= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v %= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator&=(this T, R) noexcept requires requires(T::value_type x) { x &= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v &= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator|=(this T, R) noexcept requires requires(T::value_type x) { x |= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v |= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator^=(this T, R) noexcept requires requires(T::value_type x) { x ^= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v ^= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator<<=(this T, R) noexcept requires requires(T::value_type x) { x <<= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v <<= R::value; }()>{}; }
    template<constexpr param T, constexpr param R>
      constexpr auto operator>>=(this T, R) noexcept requires requires(T::value_type x) { x >>= R::value; }
        { return constant_wrapper<[] { auto v = T::value; return v >>= R::value; }()>{}; }
    template<constexpr-param T>
      constexpr auto operator++(this T) noexcept -> constant_wrapper<++Y> { return {}; }
    template<constexpr-param T>
      constexpr auto operator++(this T, int) noexcept -> constant_wrapper<Y++> { return {}; }
    template<constexpr-param T>
      constexpr auto operator--(this T) noexcept -> constant_wrapper<--Y> { return {}; }
    template<constexpr-param T>
      constexpr auto operator--(this T, int) noexcept -> constant_wrapper<Y--> { return {}; }

    template<constexpr-param T, constexpr-param R>
      constexpr auto operator+=(T, R) noexcept -> constant_wrapper<(T::value += R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator-=(T, R) noexcept -> constant_wrapper<(T::value -= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator*=(T, R) noexcept -> constant_wrapper<(T::value *= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator/=(T, R) noexcept -> constant_wrapper<(T::value /= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator%=(T, R) noexcept -> constant_wrapper<(T::value %= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator&=(T, R) noexcept -> constant_wrapper<(T::value &= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator|=(T, R) noexcept -> constant_wrapper<(T::value |= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator^=(T, R) noexcept -> constant_wrapper<(T::value ^= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator<<=(T, R) noexcept -> constant_wrapper<(T::value <<= R::value)> { return {}; }
    template<constexpr-param T, constexpr-param R>
      constexpr auto operator>>=(T, R) noexcept -> constant_wrapper<(T::value >>= R::value)> { return {}; }
  };
}

template<cw-fixed-value X, typename>
struct constant_wrapper: cw-operators {
  static constexpr const auto & value = X.data;
  using type = constant_wrapper;
  using value_type = typename decltype(X)::type;

  template<constexpr param R>
    constexpr auto operator=(R) const noexcept requires requires(value_type x) { x = R::value; }
      { return constant_wrapper<[] { auto v = value; return v = R::value; }()>{}; }
  template<constexpr-param R>
    constexpr auto operator=(R) const noexcept -> constant_wrapper<X = R::value> { return {}; }

  constexpr operator decltype(auto)() const noexcept { return value; }
};
```

4388. Align new definition of `va_start` with C23

Section: 17.14.2 [cstdarg.syn] **Status:** Immediate **Submitter:** Jakub Jelinek **Opened:** 2025-10-01 **Last modified:** 2025-11-06

Priority: 1

View other active issues in [cstdarg.syn].

View all other issues in [cstdarg.syn].

Discussion:

P3348R4 changed the `va_start` macro to match C23, but the following wording from C is not present in C++:

> If any additional arguments expand to include unbalanced parentheses, or a preprocessing token that does not convert to a token, the behavior is undefined.

The importance of that wording was not realized during review of P3348R4. The wording is intended to ensure that any discarded arguments to `va_start` are actually lexable by the compiler, rather than containing unbalanced parentheses or brackets. It also makes the following undefined:

```
#define BAD ); format_disk(
va_start(ap, BAD);
```

[2025-10-14; Reflector poll]

Set priority to 1 after reflector poll.

[Kona 2025-11-05; LWG had questions about requiring some cases to be ill-formed.]

The submitter clarified that it would constrain implementations (effectively requiring `va_start` to be implemented as a magic keyword in the preprocessor, in order to be able to diagnose misuses when preprocessing separately from compilation).

Core approved the new wording.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 17.14.2 [cstdarg.syn] as indicated:

> (1.2) — If more than one argument is present for `va_start` and any of the second or subsequent arguments expands to include unbalanced parentheses, or a preprocessing token that does not convert to a token, the program is ill-formed, no diagnostic required. The preprocessing tokens comprising the second and subsequent arguments to `va_start` (if any) are discarded. [*Note 1*: `va_start` accepts a second argument for compatibility with prior revisions of C++. — *end note*]

4396. Improve `inplace_vector(from_range_t, R&& rg)`

Section: 23.2.4 [sequence.reqmts], 23.3.16.2 [inplace.vector.cons] **Status:** Immediate **Submitter:** Hewill Kang **Opened:** 2025-10-01 **Last modified:** 2025-11-06

Priority: 3

View other active issues in [sequence.reqmts].

View all other issues in [sequence.reqmts].

Discussion:

Consider:

```
std::array<int, 42> a;
std::inplace_vector<int, 5> v(std::from_range, a);
```

The above throws `std::bad_alloc` at runtime because the size of `array` is larger than capacity of `inplace_vector`. However, we should reject it at compile time since the `array` size is a constant expression.

Given that we do a lot of compile-time size checking in `<simd>`, it's worth applying that here as well. Compile-time errors are better than runtime ones.

[2025-10-22; Reflector poll. Status changed: New → LEWG and P3.]

General support for change, after LEWG approval.

Suggestion was made that this could be extended to all containers, but is unlikely to be triggred in real word, as it requires ranges with static size greater than `size_t(-1)`.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 23.2.4 [sequence.reqmts] as indicated:

```
a.assign_range(rg)
```

-60- *Result*: `void`

-61- *Mandates*: `assignable_from<T&, ranges::range_reference_t<R>>` is modeled. For `inplace_vector`, if `ranges::size(rg)` is a constant expression then `ranges::size(rg) ≤ a.max_size()`.

2. Modify 23.3.16.2 [inplace.vector.cons] as indicated:

```
template<container-compatible-range<T> R>
  constexpr inplace_vector(from_range_t, R&& rg);
```

-?- *Mandates*: If `ranges::size(rg)` is a constant expression then `ranges::size(rg) ≤ N`.

-9- *Effects*: Constructs an `inplace_vector` with the elements of the range `rg`.

-10- *Complexity*: Linear in `ranges::distance(rg)`.

4420. §[simd] conversions (constructor, load, stores, gather, and scatter) are incorrectly constrained for `<stdfloat>` types

Section: 29.10 [simd] **Status:** Immediate **Submitter:** Matthias Kretz **Opened:** 2025-10-15 **Last modified:** 2025-11-04

Priority: 1

View other active issues in [simd].

View all other issues in [simd].

Discussion:

Addresses DE-288 and DE-285

29.10.8.7 [simd.loadstore] `unchecked_store` and `partial_store` are constrained with `indirectly_writable` in such a way that `basic_vec`'s `value_type` must satisfy `convertible_to<`*`range value-type`*`>`. But that is not the case, e.g. for `float → float16_t` or `double → float32_t`. However, if `simd::flag_convert` is passed, these conversions were intended to work. The implementation thus must `static_cast` the `basic_vec` values to the range's value-type before storing to the range.

`unchecked_store(vec<float>, span<complex<float16_t>>, flag_convert)` does not work for a different reason. The `complex(const float16_t&, const float16_t&)` constructor simply does not allow construction from `float`, irrespective of using implicit or explicit conversion. The only valid conversion from `float → complex<float16_t>` is via an extra step through `complex<float16_t>::value_type`. This issue considers it a defect of `complex` that an explicit conversion from `float → complex<float16_t>` is ill-formed and therefore no workaround/special case is introduced.

Conversely, the conversion constructor in 29.10.7.2 [simd.ctor] does not reject conversion from `vec<complex<float>, 4>` to `vec<float, 4>`. I.e. `convertible_to<vec<complex<float>, 4>, vec<float, 4>>` is `true`, which is a lie. This is NB comment DE-288. However, the NB comment's proposed resolution is too strict, in that it would disallow conversion from `float` to `float16_t`.

The conversion/load from static-sized range constructor in 29.10.7.2 [simd.ctor] has a similar problem:

```
convertible_to<array<std::string, 4>, vec<int, 4>> is true
```

but when fixing this

```
vec<float16_t, 4>(array<float, 4>, flag_convert)
```

must continue to be valid.

`unchecked_load` and `partial_load` in 29.10.8.7 [simd.loadstore] currently *Mandate* the range's value-type to be vectorizable, but converting loads from `complex<float>` to `float` are not covered. It is unclear what a conversion from `complex<float>` to `float` should do, so it needs to be added (again without breaking `float → float16_t`).

29.10.8.11 [simd.permute.memory] is analogous to 29.10.8.7 [simd.loadstore] and needs equivalent constraints.

29.10.7.2 [simd.ctor] p2 requires `constructible_from<U>`, which makes explicit construction of `vec<float16_t>` from `float` ill-formed. For consistency this should also be constrained with *`explicitly-convertible-to`*.

[2025-10-22; Reflector poll.]

Set priority to 1 after reflector poll.

We also need to update *Effects*. There are more places in 29.10 [simd] where `float` to `float16_t` and similar conversion are not supported.

It was pointed out that similar issues happen for `complex<float16_t>`. There seem to be mismatch between language initialization rules and the intended usage based on library API.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. In 29.10.3 [simd.syn] and 29.10.8.7 [simd.loadstore] replace all occurrences of

   ```
   indirectly_writable<ranges::iterator_t<R>, T>
   ```

   with

   ```
   indirectly_writable<ranges::iterator_t<R>, ~~T~~ranges::range_value_t<R>>
   ```

   and all occurrences of

   ```
   indirectly_writable<I, T>
   ```

   with

   ```
   indirectly_writable<I, ~~T~~iter_value_t<I>>
   ```

2. Modify 29.10.8.7 [simd.loadstore] as indicated:

   ```
   template<class T, class Abi, ranges::contiguous_range R, class... Flags>
     requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
     constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r, flags<Flags...> f = {});
   […]
   template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
     requires indirectly_writable<I, T>
     constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
       const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
   ```

   -11- Let […]

   -?- *Constraints*: The expression `static_cast<ranges::range_value_t<R>>(x)` where `x` is an object of type `T` is well-formed.

-12- *Mandates*: If `ranges::size(r)` is a constant expression then `ranges::size(r) ≥ simd-size-v<T, Abi>`.

[…]

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r, flags<Flags...> f = {});
[…]
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
```

-15- Let […]

-?- *Constraints*: The expression `static_cast<iter_value_t<I>>(x)` where `x` is an object of type `T` is well-formed.

-16- *Mandates*: […]

[2025-10-22; Matthias Kretz improves discussion and provides new wording]

[Kona 2025-11-04; Also resolves LWG 4393(i).]

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 29.10.2 [simd.expos] as indicated:

```
[…]

template<class T>
  concept constexpr-wrapper-like =                    // exposition only
    […]
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value>::value;

template<class From, class To>
  concept explicitly-convertible-to =                 // exposition-only
    requires {
      static_cast<To>(declval<From>());
    };

template<class T> using deduced-vec-t = see below; // exposition only
[…]
```

2. Modify 29.10.3 [simd.syn] as indicated:

```
[…]
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
                                 flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
                                 const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
                                 iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
                                 iter_difference_t<I> n, const typename basic_vec<T, Abi>::mask_type& mask,
                                 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
                                 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
                                 const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});

template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
                               flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(
    const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(
    const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
```

```
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
                               flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
[…]
```

3. Modify 29.10.7.2 [simd.ctor] as indicated:

```
template<class U> constexpr explicit(see below) basic_vec(U&& value) noexcept;
```

-1- Let `From` denote the type `remove_cvref_t<U>`.

-2- *Constraints*: ~~`value_type`~~`U` satisfies ~~`constructible_from<U>`~~*`explicitly-convertible-to`*`<value_type>`.

[…]

```
template<class U, class UAbi>
  constexpr explicit(see below) basic_vec(const basic_vec<U, UAbi>& x) noexcept;
```

-5- *Constraints*:

(5.1) — *`simd-size-v`*`<U, UAbi> == size()` is `true`, and

(5.2) — `U` satisfies *`explicitly-convertible-to`*`<T>`.

[…]

```
template<class R, class... Flags>
  constexpr basic_vec(R&& r, flags<Flags...> = {});
template<class R, class... Flags>
  constexpr basic_vec(R&& r, const mask_type& mask, flags<Flags...> = {});
```

-12- Let `mask` be `mask_type(true)` for the overload with no `mask` parameter.

-13- *Constraints*:

(13.1) — `R` models `ranges::contiguous_range` and `ranges::sized_range`,

(13.2) — `ranges::size(r)` is a constant expression, ~~and~~

(13.3) — `ranges::size(r)` is equal to `size()`, and

(13.?) — `ranges::range_value_t<R>` is a vectorizable type and satisfies *`explicitly-convertible-to`*`<T>`.

-14- *Mandates*:

~~(14.1) — `ranges::range_value_t<R>` is a vectorizable type, and~~

~~(14.2) — if~~If the template parameter pack `Flags` does not contain *`convert-flag`*, then the conversion from `ranges::range_value_t<R>` to `value_type` is value-preserving.

4. Modify 29.10.8.7 [simd.loadstore] as indicated:

```
template<class V = see below , ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R>
  constexpr V partial_load(R&& r, flags<Flags...> f = {});
template<class V = see below , ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R>
  constexpr V partial_load(R&& r, const typename V::mask_type& mask, flags<Flags...> f = {});
template<class V = see below , contiguous_iterator I, class... Flags>
  constexpr V partial_load(I first, iter_difference_t<I> n, flags<Flags...> f = {});
template<class V = see below , contiguous_iterator I, class... Flags>
  constexpr V partial_load(I first, iter_difference_t<I> n, const typename V::mask_type& mask,
                           flags<Flags...> f = {});
template<class V = see below , contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  constexpr V partial_load(I first, S last, flags<Flags...> f = {});
template<class V = see below , contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  constexpr V partial_load(I first, S last, const typename V::mask_type& mask,
                           flags<Flags...> f = {});
```

-6- Let

(6.1) — `mask` be `V::mask_type(true)` for the overloads with no `mask` parameter;

(6.2) — `R` be `span<const iter_value_t<I>>` for the overloads with no template parameter `R`;

(6.3) — `r` be `R(first, n)` for the overloads with an `n` parameter and `R(first, last)` for the overloads with a `last` parameter~~.~~;

(6.?) — `T` be `typename V::value_type`.

-7- *Mandates*:

(7.1) — `ranges::range_value_t<R>` is a vectorizable type and satisfies *`explicitly-convertible-to`*`<T>`,

(7.2) — `same_as<remove_cvref_t<V>, V>` is `true`,

(7.3) — `V` is an enabled specialization of `basic_vec`, and

(7.4) — if the template parameter pack `Flags` does not contain *`convert-flag`*, then the conversion from `ranges::range_value_t<R>` to `V::value_type` is value-preserving.

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
                                 flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
                                 const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
                                 iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
                                 iter_difference_t<I> n, const typename basic_vec<T, Abi>::mask_type& mask,
                                 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
                                 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
                                 const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
```

-11- Let […]

[…]

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
                               flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
  requires ranges::sized_range<R> && indirectly_writable<ranges::iterator_t<R>, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(
    const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(
    const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
                               flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
  requires indirectly_writable<I, T>
  constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
    const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
```

-15- Let […]

-?- *Constraints*:

(?.1) — `ranges::iterator_t<R>` models `indirectly_writable<ranges::range_value_t<R>>`, and

(?.2) — `T` satisfies *`explicitly-convertible-to`*`<ranges::range_value_t<R>>`

-16- *Mandates*: […]

-17- *Preconditions*: […]

-18- *Effects*: For all *`i`* in the range of `[0, basic_vec<T, Abi>::size())`, if `mask[`*`i`*`] && `*`i`*` < ranges::size(r)` is `true`, evaluates `ranges::data(r)[`*`i`*`] = static_cast<ranges::range_value_t<R>>(v[`*`i`*`])`.

5. Modify 29.10.8.11 [simd.permute.memory] as indicated:

```
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
  requires ranges::sized_range<R>
  constexpr V partial_gather_from(R&& in, const I& indices, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
  requires ranges::sized_range<R>
  constexpr V partial_gather_from(R&& in, const typename I::mask_type& mask,
                                  const I& indices, flags<Flags...> f = {});
```

-5- Let: […]

-?- *Constraints:* `ranges::range_value_t<R>` is a vectorizable type and satisfies *`explicitly-convertible-to`*`<T>`.

-6- *Mandates*: […]

[…]

```
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
  requires ranges::sized_range<R>
  constexpr void
  partial_scatter_to(const V& v, R&& out, const I& indices, flags<Flags...> f = {});
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
  requires ranges::sized_range<R>
  constexpr void partial_scatter_to(const V& v, R&& out, const typename I::mask_type& mask,
                                    const I& indices, flags<Flags...> f = {});
```

-13- Let `mask` be `I::mask_type(true)` for the overload with no `mask` parameter.

-14- *Constraints*:

(14.1) — `V::size() == I::size()` is `true`,

(14.2) — `ranges::iterator_t<R>` models `indirectly_writable<ranges::range_value_t<R>>`, and

(14.3) — `typename V::value_type` satisfies *`explicitly-convertible-to`*`<ranges::range_value_t<R>>`.

[…]

-17- *Effects*: For all i in the range `[0, I::size())`, if `mask[`i`] && (indices[`i`] < ranges::size(out))` is `true`, evaluates `ranges::data(out)[indices[`i`]] = static_cast<ranges::range_value_t<R>>(v[`i`])`.

4424. `meta::define_aggregate` should require a class type

Section: 21.4.16 [meta.reflection.define.aggregate] **Status:** Immediate **Submitter:** Jakub Jelinek **Opened:** 2025-10-20 **Last modified:** 2025-11-04

Priority: 1

View other active issues in [meta.reflection.define.aggregate].

View all other issues in [meta.reflection.define.aggregate].

Discussion:

Addresses US 125-188

The `meta::define_aggregate` function doesn't say what happens if C does not represent a class type.

It's also unclear whether it should work with aliases to class types, e.g.

```
struct S; using A = S; ... meta::define_aggregate(^^A, {});
```

And what happens if you try to define a cv-qualified type:

```
struct S; meta::define_aggregate(^^const S, {});
```

Should this be an error, or inject a definition of the unqualified type?

[2025-10-23; Reflector poll.]

Set priority to 1 after reflector poll.

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 21.4.16 [meta.reflection.define.aggregate] as indicated:

-7- Let C be the class represented by `dealias(class_type)` and r_K be the K^{th} reflection value in `mdescrs`. For every r_K in `mdescrs`, let $(T_K, N_K, A_K, W_K, NUA_K)$ be the corresponding data member description represented by r_K.

-8- *Constant when*:

(8.?) — `dealias(class_type)` represents a class type;
(8.1) — C is incomplete from every point in the evaluation context;

[2025-10-24; LWG telecon. Jonathan updates wording]

Make a minimal change for now, can add support for aliases later.

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.16 [meta.reflection.define.aggregate] as indicated:

-7- Let C be the ~~class~~ type represented by `class_type` and r_K be the K^{th} reflection value in `mdescrs`. For every r_K in `mdescrs`, let $(T_K, N_K, A_K, W_K, NUA_K)$ be the corresponding data member description represented by r_K.

-8- *Constant when*:

(8.?) — `class_type` represents a cv-unqualified class type;
(8.1) — *C* is incomplete from every point in the evaluation context;

4427. `meta::dealias` needs to work with things that aren't entities

Section: 21.4.7 [meta.reflection.queries] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-10-24 **Last modified:** 2025-11-04

Priority: Not Prioritized

View other active issues in [meta.reflection.queries].

View all other issues in [meta.reflection.queries].

Discussion:

Addresses US 99-205

Several uses of `dealias` assume that it can be used with reflections that represent direct base class relationships, which are not entities. The spec for `dealias` says that such uses should fail with an exception.

In the 2025-10-24 LWG telecon it was agreed that `dealias` should just be the identity function for non-entities.

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.7 [meta.reflection.queries] as indicated:

```
consteval info dealias(info r);
```

-49- *Returns*: If `r` represents an entity, then a ~~A~~ reflection representing the underlying entity of what `r` represents. Otherwise, `r`.

[*Example 5*:
...
— *end example*]

~~-50- *Throws*: `meta::exception` unless `r` represents an entity.~~

4428. Metafunctions should not be defined in terms of constant subexpressions

Section: 21.4.9 [meta.reflection.access.queries], 21.4.18 [meta.reflection.annotation], 21.4.15 [meta.reflection.array] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-10-24 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [meta.reflection.access.queries].

View all other issues in [meta.reflection.access.queries].

Discussion:

Addresses US 102-209

"is a constant (sub)expression" is incorrect now that errors are reported via exceptions.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 21.4.9 [meta.reflection.access.queries] as indicated:

```
consteval bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
```

-5- *Returns*: `true` if `is_accessible(R, ctx)` is `false` for any *R* in `nonstatic_data_members_of(r, access_context::unchecked())`. Otherwise, `false`.

-6- *Throws*: `meta::exception` unless

(6.1) — the evaluation of `nonstatic_data_members_of(r, access_context::unchecked())` ~~is a constant subexpression~~ would not exit via an exception and
(6.2) — `r` does not represent a closure type.

```
consteval bool has_inaccessible_bases(info r, access_context ctx);
```

-5- *Returns*: `true` if `is_accessible(R, ctx)` is `false` for any *R* in `bases_of(r, access_context::unchecked())`. Otherwise, `false`.

-6- *Throws*: `meta::exception` unless the evaluation of `bases_of(r, access_context::unchecked())` ~~is a constant subexpression~~ would not exit via an exception.

2. Modify 21.4.18 [meta.reflection.annotation] as indicated:

```
consteval vector<info> annotations_of_with_type(info item, info type);
```

-4- *Returns*: A `vector` containing each element `e` of `annotations_of(item)` where `remove_const(type_of(e)) == remove_const(type)` is `true`, preserving their order.

-5- *Throws*: `meta::exception` unless

(5.1) — the evaluation of `annotations_of(item)` ~~is a constant subexpression~~ would not exit via an exception and
(5.2) — `dealias(type)` represents a type that is complete from some point in the evaluation context.

3. Modify 21.4.15 [meta.reflection.array] as indicated:

```
template<ranges::input_range R>
  consteval info reflect_constant_array(R&& r);
```

-8- Let `T` be `ranges::range_value_t<R>`.

-9- *Mandates*: `T` is a structural type (13.2 [temp.param]), `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`, and `is_copy_constructible_v<T>` is `true`.

-10- Let V be the pack of values of type `info` of the same size as `r`, where the i^{th} element is `reflect_constant(e`$_i$`)`, where `e`$_i$ is the i^{th} element of `r`.

-11- Let P be

(11.1) — If `sizeof...(`V`) > 0` is `true`, then the template parameter object (13.2 [temp.param]) of type `const T[sizeof...(`V`)]` initialized with `{[:V:]...}`.
(11.2) — Otherwise, the template parameter object of type `array<T, 0>` initialized with `{}`.

-12- *Returns*: `^^`P.

-13- *Throws*: `meta::exception` unless the evaluation of `reflect_constant(e)` ~~is a constant subexpression~~ would not exit via an exception for every element `e` of `r`.

[Kona 2025-11-06; Jonathan removes change to 21.4.15 [meta.reflection.array] that was handled by LWG 4432(i).]

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.9 [meta.reflection.access.queries] as indicated:

```
consteval bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
```

-5- *Returns*: `true` if `is_accessible(`R`, ctx)` is `false` for any R in `nonstatic_data_members_of(r, access_context::unchecked())`. Otherwise, `false`.

-6- *Throws*: `meta::exception` ~~unless~~ if

(6.1) — the evaluation of `nonstatic_data_members_of(r, access_context::unchecked())` ~~is a constant subexpression~~ would exit via an exception ~~and~~ or
(6.2) — `r` ~~does not represent~~ represents a closure type.

```
consteval bool has_inaccessible_bases(info r, access_context ctx);
```

-7- *Returns*: `true` if `is_accessible(`R`, ctx)` is `false` for any R in `bases_of(r, access_context::unchecked())`. Otherwise, `false`.

-8- *Throws*: `meta::exception` ~~unless~~ if the evaluation of `bases_of(r, access_context::unchecked())` ~~is a constant subexpression~~ would exit via an exception.

2. Modify 21.4.18 [meta.reflection.annotation] as indicated:

```
consteval vector<info> annotations_of_with_type(info item, info type);
```

-4- *Returns*: A `vector` containing each element `e` of `annotations_of(item)` where `remove_const(type_of(e)) == remove_const(type)` is `true`, preserving their order.

-5- *Throws*: `meta::exception` unless

(5.1) — the evaluation of `annotations_of(item)` ~~is a constant subexpression~~ would not exit via an exception and
(5.2) — `dealias(type)` represents a type that is complete from some point in the evaluation context.

4429. `meta::alignment_of` should exclude data member description of bit-field

Section: 21.4.11 [meta.reflection.layout] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-10-24 **Last modified:** 2025-11-04

Priority: Not Prioritized

Discussion:

Addresses US 109-170

21.4.11 [meta.reflection.layout] p#8 This should similarly disallow data member descriptions of bit-fields.

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.11 [meta.reflection.layout] as indicated:

```
consteval size_t alignment_of(info r);
```

-7- *Returns*: […]

-8- *Throws*: `meta::exception` unless all of the following conditions are met:

(8.1) — `dealias(r)` is a reflection of a type, object, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description *(T,N,A,W,NUA)* (11.4.1 [class.mem.general]) where *W* is ⊥.
(8.2) — If `dealias(r)` represents a type, then `is_complete_type(r)` is true.

4430. `from_chars` should not parse `"0b"` base prefixes

Section: 28.2.3 [charconv.from.chars] **Status:** Immediate **Submitter:** Jan Schultke **Opened:** 2025-10-20 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [charconv.from.chars].

View all other issues in [charconv.from.chars].

Discussion:

C23 added support for the `"0b"` and `"0B"` base prefix to `strtol`, and since the wording of `from_chars` for integers is based on `strol`, this inadvertently added support for parsing `"0b"` prefixes to `from_chars`.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 28.2.3 [charconv.from.chars] as indicated:

```
constexpr from_chars_result from_chars(const char* first, const char* last,
                                       integer-type& value, int base = 10);
```

-2- *Preconditions*: `base` has a value between 2 and 36 (inclusive).

-3- *Effects*: The pattern is the expected form of the subject sequence in the "C" locale for the given nonzero base, as described for `strtol`, except that no `"0b"` or `"0B"` prefix shall appear if the value of `base` is 2, no `"0x"` or `"0X"` prefix shall appear if the value of `base` is 16, and except that `'-'` is the only sign that may appear, and only if `value` has a signed type.

-4- *Throws*: Nothing.

4431. Parallel `std::ranges::destroy` should allow exceptions

Section: 20.2.2 [memory.syn] **Status:** Immediate **Submitter:** Ruslan Arutyunyan **Opened:** 2025-10-24 **Last modified:** 2025-11-08

Priority: Not Prioritized

View all other issues in [memory.syn].

Discussion:

The serial `std::ranges::destroy` algorithm is marked as `noexcept`. However, the parallel counterpart should not be marked `noexcept`.

While we generally don't expect any exceptions from the `destroy` algorithm when called with the standard execution policies (`seq`, `unseq`, `par`, `par_unseq`), the implementation-defined policies for parallel algorithms are allowed by the C++ standard, and it is up to the particular execution policy to decide which exceptions can be thrown from parallel algorithms.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 20.2.2 [memory.syn], header <memory> synopsis, as indicated:

[*Drafting note*: There are no further prototype definitions for the affected *execution-policy* overloads in 26.11.9 [specialized.destroy].]

```
[…]
// 26.11.9, 26.11.9 [specialized.destroy]
template<class T>
  constexpr void destroy_at(T* location);                                    // freestanding
template<class NoThrowForwardIterator>
  constexpr void destroy(NoThrowForwardIterator first,                       // freestanding
                         NoThrowForwardIterator last);
template<class ExecutionPolicy, class NoThrowForwardIterator>
  void destroy(ExecutionPolicy&& exec,                                       // freestanding-deleted,
               NoThrowForwardIterator first,                                 // see 26.3.5 [algorithms.parallel.overloads]
               NoThrowForwardIterator last);
template<class NoThrowForwardIterator, class Size>
  constexpr NoThrowForwardIterator destroy_n(NoThrowForwardIterator first, // freestanding
                                             Size n);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
  NoThrowForwardIterator destroy_n(ExecutionPolicy&& exec,                   // freestanding-deleted,
                                   NoThrowForwardIterator first, Size n);  // see 26.3.5 [algorithms.parallel.overloads]
namespace ranges {
  template<destructible T>
    constexpr void destroy_at(T* location) noexcept;                         // freestanding

  template<nothrow-input-iterator I, nothrow-sentinel-for <I> S>
    requires destructible<iter_value_t<I>>
      constexpr I destroy(I first, S last) noexcept;                         // freestanding
  template<nothrow-input-range R>
    requires destructible<range_value_t<R>>
      constexpr borrowed_iterator_t<R> destroy(R&& r) noexcept;              // freestanding

  template<nothrow-input-iterator I>
    requires destructible<iter_value_t<I>>
    constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept;         // freestanding

  template<execution-policy Ep, nothrow-random-access-iterator I,
           nothrow-sized-sentinel-for <I> S>
    requires destructible<iter_value_t<I>>
      I destroy(Ep&& exec, I first, S last) noexcept;                         // freestanding-deleted,
                                                                             // see 26.3.5 [algorithms.parallel.overloads]
  template<execution-policy Ep, nothrow-sized-random-access-range R>
    requires destructible<range_value_t<R>>
      borrowed_iterator_t<R> destroy(Ep&& exec, R&& r) noexcept;             // freestanding-deleted,
                                                                             // see 26.3.5 [algorithms.parallel.overloads]
  template<execution-policy Ep, nothrow-random-access-iterator I>
    requires destructible<iter_value_t<I>>
      I destroy_n(Ep&& exec, I first, iter_difference_t<I> n) noexcept;      // freestanding-deleted,
                                                                             // see 26.3.5 [algorithms.parallel.overloads]
}
[…]
```

4432. Clarify element initialization for `meta::reflect_constant_array`

Section: 21.4.3 [meta.define.static] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-10-27 **Last modified:** 2025-11-04

Priority: Not Prioritized

Discussion:

Addresses US 120-181 and US 121-182

21.4.15 [meta.reflection.array] p10 Clarify e_i type. It is not clear what e_i is when proxy references are involved.

21.4.15 [meta.reflection.array] Clarify copy-initialization vs. direct-initialization use The initialization of `P` uses copy-initialization but the Mandates clause uses direct-initialization.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 21.4.3 [meta.define.static] as indicated:

```
template<ranges::input_range R>
  consteval info reflect_constant_array(R&& r);
```

-8- Let `T` be `ranges::range_value_t<R>`.

-9- *Mandates*: `T` is a structural type (13.2 [temp.param]), `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`, and ~~`is_copy_constructible_v<T>` is `true`~~ `T satisfies copy_constructible`.

-10- Let `V` be the pack of values of type info of the same size as `r`, where the i^{th} element is `reflect_constant(`~~e_i~~`static_cast<T>(`$*it_i$`))`, where ~~e_i~~ it_i is an iterator to the i^{th} element of `r`.

[…]

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.3 [meta.define.static] as indicated:

```
template<ranges::input_range R>
  consteval info reflect_constant_array(R&& r);
```

-8- Let `T` be `ranges::range_value_t<R>` and e_i be `static_cast<T>(*`it_i`)`, where it_i is an iterator to the i^{th} element of `r`.

-9- *Mandates*: `T` is a structural type (13.2 [temp.param]), `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`, and ~~`is_copy_constructible_v<T>` is `true`~~ `T` satisfies `copy_constructible`.

-10- Let `V` be the pack of values of type `info` of the same size as `r`, where the i^{th} element is `reflect_constant(`e_i`)`~~, where e_i is an iterator to the i^{th} element of `r`~~.

[…]

-13- Throws: Any exception thrown by the evaluation of any e_i, or `meta::exception` ~~unless~~if evaluation of any `reflect_constant(`~~`e`~~e_i`)` would exit via an exception ~~is a constant subexpression for every element `e` of `r`~~.

4433. Incorrect query for C language linkage

Section: 21.4.7 [meta.reflection.queries] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-10-27 **Last modified:** 2025-11-04

Priority: Not Prioritized

View other active issues in [meta.reflection.queries].

View all other issues in [meta.reflection.queries].

Discussion:

Addresses US 97-203

21.4.7 [meta.reflection.queries] Language linkage is a property of functions, variables, and function types (6.7 [basic.link]), not of names.

[The wording below contains a drive-by fix for a misapplication of P2996R13]

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.7 [meta.reflection.queries] as indicated:

```
consteval bool has_internal_linkage(info r);
consteval bool has_module_linkage(info r);
consteval bool has_external_linkage(info r);
~~consteval bool has_c_language_linkage(info r);~~
consteval bool has_linkage(info r);
```

-25- *Returns*: `true` if `r` represents a variable, function, type, template, or namespace whose name has internal linkage, module linkage, ~~C language~~external linkage, or any linkage, respectively (6.7 [basic.link]). Otherwise, `false`.

```
consteval bool has_c_language_linkage(info r);
```

-??- *Returns*: `true` if `r` represents a variable, function, or function type with C language linkage. Otherwise, `false`.

4434. `meta::is_accessible` does not need to consider incomplete *D*

Section: 21.4.9 [meta.reflection.access.queries] **Status:** Immediate **Submitter:** Jakub Jelinek **Opened:** 2025-10-27 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [meta.reflection.access.queries].

View all other issues in [meta.reflection.access.queries].

Discussion:

21.4.9 [meta.reflection.access.queries] says that `is_accessible(r, ctx)` throws if:

> `r` represents a direct base class relationship (*D*,*B*) for which *D* is incomplete.

However, the only way to get access to a direct base relationship is through `bases_of`/`subobjects_of` and those throw if the class is incomplete, so I don't see how an `is_base` reflection could have ever incomplete *D*.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.9 [meta.reflection.access.queries] as indicated:

-4- *Throws*: `meta::exception` if~~:~~

~~(4.1) —~~ `r` represents a class member for which *`PARENT-CLS`*`(r)` is an incomplete class ~~or~~.
~~(4.2) — `r` represents a direct base class relationship (*D*,*B*) for which *D* is incomplete.~~

4435. `meta::has_identifier` doesn't handle all types

Section: 21.4.6 [meta.reflection.names] **Status:** Immediate **Submitter:** Jakub Jelinek **Opened:** 2025-10-27 **Last modified:** 2025-11-07

Priority: 2

Discussion:

The wording for `meta::has_identifier` doesn't specify what it returns for `^^int` or `^^void` or `^^Enum`.

[2025-10-29; Reflector poll.]

Set priority to 2 after reflector poll.

Move bullet point for aliases before bullet for types. Add "cv-unqualified" to class type and enumeration type. Simplify "`!has_template_arguments()` is `true`" to "`has_template_arguments()` is `false`".

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.6 [meta.reflection.names] as indicated:

```
consteval bool has_identifier(info r);
```

-1- *Returns*:

(1.1) — If `r` represents an entity that has a typedef name for linkage purposes (9.2.4 [dcl.typedef]), then `true`.
(1.2) — Otherwise, if `r` represents an unnamed entity, then `false`.
(1.?) — Otherwise, if `r` represents a type alias, then `!has_template_arguments(r)`.
(1.3) — Otherwise, if `r` represents a type, then `true` if ~~class type, then `!has_template_arguments(r)`.~~
  (1.3.1) — `r` represents a cv-unqualified class type and `has_template_arguments(r)` is `false`, or
  (1.3.2) — `r` represents a cv-unqualified enumeration type.
Otherwise, `false`.
(1.4) — Otherwise, if `r` represents a function, then `true` if `has_template_arguments(r)` is `false` and the function is not a constructor, destructor, operator function, or conversion function. Otherwise, `false`.
(1.5) — Otherwise, if `r` represents a template, then `true` if `r` does not represent a constructor template, operator function template, or conversion function template. Otherwise, `false`.
(1.6) — Otherwise, if `r` represents the i^{th} parameter of a function F that is an (implicit or explicit) specialization of a templated function T and the i^{th} parameter of the instantiated declaration of T whose template arguments are those of F would be instantiated from a pack, then `false`.
(1.7) — Otherwise, if `r` represents the parameter P of a function F, then let S be the set of declarations, ignoring any explicit instantiations, that precede some point in the evaluation context and that declare either F or a templated function of which F is a specialization; `true` if
  (1.7.1) — there is a declaration D in S that introduces a name N for either P or the parameter corresponding to P in the templated function that D declares and
  (1.7.2) — no declaration in S does so using any name other than N.
Otherwise, `false`.
(1.8) — Otherwise, if `r` represents a variable, then `false` if the declaration of that variable was instantiated from a function parameter pack. Otherwise, `!has_template_arguments(r)`.
(1.9) — Otherwise, if `r` represents a structured binding, then `false` if the declaration of that structured binding was instantiated from a structured binding pack. Otherwise, `true`.
~~(1.10) — Otherwise, if `r` represents a type alias, then `!has_template_arguments(r)`.~~
(1.11) — Otherwise, if `r` represents an enumerator, non-static-data member, namespace, or namespace alias, then `true`.
(1.12) — Otherwise, if `r` represents a direct base class relationship, then `has_identifier(type_of(r))`.
(1.13) — Otherwise, `r` represents a data member description (T,N,A,W,NUA) (11.4.1 [class.mem.general]); `true` if N is not $\perp$. Otherwise, `false`.

4438. Bad expression in [exec.when.all]

Section: 33.9.12.12 [exec.when.all] **Status:** Immediate **Submitter:** Eric Niebler **Opened:** 2025-10-30 **Last modified:** 2025-11-06

Priority: Not Prioritized

View all other issues in [exec.when.all].

Discussion:

Addresses US 220-344 and US 224-342

33.9.12.12 [exec.when.all] p5 reads as follows:

> -5- Let *make-when-all-env* be the following exposition-only function template:
>
> ```
> template<class Env>
>   constexpr auto make-when-all-env(inplace_stop_source& stop_src,        // exposition only
>                                    Env&& env) noexcept {
>   return see below;
> }
> ```
>
> Returns an object e such that
>
> (5.1) — `decltype(e)` models *queryable*, and
> (5.2) — `e.query(get_stop_token)` is expression-equivalent to `state.stop-src.get_token()`, and
> (5.3) — given a query object `q` with type other than *cv* `stop_token_t` and whose type satisfies *forwarding-query*, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`.

The problem is with "`state.stop-src.get_token()`" in bullet (5.2). There is no `state` object here. This expression should be `stop_src.get_token()`.

[Kona 2025-11-04; add edits to address NB comments.]

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.12.12 [exec.when.all] as indicated:

> -5- Let *make-when-all-env* be the following exposition-only function template:
>
> ```
> template<class Env>
>   constexpr auto make-when-all-env(inplace_stop_source& stop_src,        // exposition only
>                                    Env&& env) noexcept; ~~{~~
> ~~  return see below;~~
> ~~}~~
> ```
>
> -?- *Returns*: An ~~Returns and~~ object e such that
>
> (5.1) — `decltype(e)` models *queryable*, and
> (5.2) — `e.query(get_stop_token)` is expression-equivalent to ~~`state.stop-src`~~`stop_src.get_token()`, and
> (5.3) — given a query object `q` with type other than *cv* `get_stop_token_t` ~~and whose type satisfies *forwarding-query*~~, `e.query(q)` is expression-equivalent to ~~`get_env(rcvr)`~~`env.query(q)` if the type of `q` satisfies *forwarding-query*, and ill-formed otherwise.

4439. `std::optional<T&>::swap` possibly selects ADL-found `swap`

Section: 22.5.4.4 [optional.ref.swap] **Status:** Immediate **Submitter:** Jiang An **Opened:** 2025-10-31 **Last modified:** 2025-11-07

Priority: Not Prioritized

Discussion:

Currently, 22.5.4.4 [optional.ref.swap] p1 specifies an "unqualified" `swap` call, which possibly selects an ADL-found swap function due to 16.4.2.2 [contents] and 16.4.4.3 [swappable.requirements].

It's unlike to be intentional to call ADL-found `swap` on pointers (given `ranges::swap` doesn't), and the unconditional `noexcept` also suggests that user-provided `swap` functions shouldn't interfere with `optional<T&>::swap`.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 22.5.4.4 [optional.ref.swap] as indicated:

```
constexpr void swap(optional& rhs) noexcept;
```

> -1- *Effects*: Equivalent to: `std::swap(val, rhs.val)`.

4440. Forward declarations of entities need also in entries

Section: 17.3.2 [version.syn] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-03 **Last modified:** 2025-11-04

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

Addresses US 65-116

There are forward declarations of entities from `<spanstream>` and `<syncstream>` in `<iosfwd>` so their feature macros should be added to that header too. Proposed change: Add `<iosfwd>` to the "also in" entries for `__cpp_lib_char8_t`, `__cpp_lib_spanstream`, and `__cpp_lib_syncbuf`.

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 17.3.2 [version.syn] as indicated:

```
#define __cpp_lib_char8_t                           201907L
  // freestanding, also in <atomic>, <filesystem>, <iosfwd>, <istream>, <limits>, <locale>, <ostream>, <string>,
  // <string_view>
[…]
#define __cpp_lib_spanstream                        202106L // also in <iosfwd>, <spanstream>
[…]
#define __cpp_lib_syncbuf                           201803L // also in <iosfwd>, <syncstream>
```

4441. `ranges::rotate` do not handle sized-but-not-sized-sentinel ranges correctly

Section: 26.7.11 [alg.rotate], 26.8.2.3 [partial.sort], 26.8.3 [alg.nth.element], 26.8.6 [alg.merge] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-03 **Last modified:** 2025-11-04

Priority: Not Prioritized

View all other issues in [alg.rotate].

Discussion:

Addresses US 161-258

These do not handle sized-but-not-sized-sentinel ranges correctly.

[Kona 2025-11-03; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.7.11 [alg.rotate] as indicated:

```
template<execution-policy Ep, sized-random-access-range R>
  requires permutable<iterator_t<R>>
  borrowed_subrange_t<R> ranges::rotate(Ep&& exec, R&& r, iterator_t<R> middle);
```

-6- *Effects* Equivalent to: `return ranges::rotate(std::forward<Ep>(exec), ranges::begin(r), middle, ~~ranges::end(r)~~ranges::begin(r) + ranges::distance(r));`

[…]

```
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
  ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
          ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);
```

-18- *Effects* Equivalent to: `return ranges::rotate(std::forward<Ep>(exec), ranges::begin(r), middle, ~~ranges::end(r)~~ranges::begin(r) + ranges::distance(r), ranges::begin(result_r), ~~ranges::end(result_r)~~ranges::begin(result_r) + ranges::distance(result_r));`

2. Modify 26.8.2.3 [partial.sort] as indicated:

```
template<execution-policy Ep, sized-random-access-range R,
         class Comp = ranges::less, class Proj = identity>
  requires sortable<iterator_t<R>, Comp, Proj>
  borrowed_iterator_t<R>
    ranges::partial_sort(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
                         Proj proj = {});
```

-7- *Effects* Equivalent to: `return ranges::partial_sort(std::forward<Ep>(exec), ranges::begin(r), middle, ~~ranges::end(r)~~ranges::begin(r) + ranges::distance(r), comp, proj);`

3. Modify 26.8.3 [alg.nth.element] as indicated:

```
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
         class Proj = identity>
  requires sortable<iterator_t<R>, Comp, Proj>
  borrowed_iterator_t<R>
    ranges::nth_element(Ep&& exec, R&& r, iterator_t<R> nth, Comp comp = {}, Proj proj = {});
```

-7- *Effects* Equivalent to: `return ranges::nth_element(std::forward<Ep>(exec), ranges::begin(r), nth, ~~ranges::end(r)~~ranges::begin(r) + ranges::distance(r), comp, proj);`

4. Modify 26.8.6 [alg.merge] as indicated:

```
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
         class Proj = identity>
  requires sortable<iterator_t<R>, Comp, Proj>
```

```
borrowed_iterator_t<R>
  ranges::inplace_merge(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
                        Proj proj = {});
```

-14- *Effects* Equivalent to: `return ranges::inplace_merge(std::forward<Ep>(exec), ranges::begin(r), middle, ~~ranges::end(r)~~ranges::begin(r) + ranges::distance(r), comp, proj);`

4442. Clarify `expr` and `fn` for `meta::reflect_object` and `meta::reflect_function`

Section: 21.4.14 [meta.reflection.result] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-04 **Last modified:** 2025-11-04

Priority: Not Prioritized

Discussion:

Addresses US 118-179

This should talk about the object/function designated by `expr`/`fn`, rather than `expr`/`fn`.

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.14 [meta.reflection.result] as indicated:

```
template<class T>
  consteval info reflect_object(T& expr);
```

-7- *Mandates*: `T` is an object type.

-8- *Returns*: A reflection of the object designated by `expr`.

-9- *Throws*: `meta::exception` ~~unless `expr` is~~ if `E` is not suitable for use as a constant template argument for a constant template parameter of type `T&` (13.4.3 [temp.arg.nontype]) , where `E` is an lvalue constant expression that computes the object that `expr` refers to.

```
template<class T>
  consteval info reflect_function(T& fn);
```

-10- *Mandates*: `T` is an function type.

-11- *Returns*: A reflection of the function designated by `fn`.

-12- *Throws*: `meta::exception` ~~unless `fn` is~~ if `F` is not suitable for use as a constant template argument for a constant template parameter of type `T&` (13.4.3 [temp.arg.nontype]) , where `F` is an lvalue constant expression that computes the function that `fn` refers to.

4443. Clean up identifier comparisons in `meta::define_aggregate`

Section: 21.4.16 [meta.reflection.define.aggregate] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-04 **Last modified:** 2025-11-04

Priority: Not Prioritized

View other active issues in [meta.reflection.define.aggregate].

View all other issues in [meta.reflection.define.aggregate].

Discussion:

Addresses US 127-190

N_K is defined as an identifier (see 11.4.1) and should not be compared with code or with string literals in bullet 8.4. Similarly, 9.5.1 should not talk about “character sequence encoded by N_K”

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.16 [meta.reflection.define.aggregate] as indicated:

```
template<reflection_range R = initializer_list<info>>
  consteval info define_aggregate(info class_type, R&& mdescrs);
```

-7- […]

-8- *Constant When*:

- -8.1- […]
- -8.2- […]
- -8.3- […]
- -8.4- for every pair (r_K, r_L) where $K<L$, if N_K is not $\perp$ and N_L is not $\perp$, then either:

- -8.4.1- ~~N_K != N_L is true~~ N_K is not the same identifier as N_L or
- -8.4.2- ~~N_K == u8"_" is true~~ N_K is the identifier (U+005F LOW LINE).

-9- *Effects*: Produces an injected declaration D (7.7 [expr.const]) that defines C and has properties as follows:

- -9.1- […]
- -9.2- […]
- -9.3- […]
- -9.4- […]
- -9.5- for every r_K, there is corresponding entity M_K belonging to the class scope of D with the following properties: $K<L$, if N_K is not $\perp$ and N_L is not $\perp$, then either:
  - -9.5.1- if N_K is $\perp$, M_K is an unnamed bit-field. Otherwise, M_K is a non-static data member whose name is the identifier ~~determined by the character sequence encoded by~~ N_K ~~in UTF-8~~.
  - -9.5.2- […]
  - -9.5.3- […]
  - -9.5.4- […]
  - -9.5.5- […]
- -9.6- […]

4444. Fix default template arguments for `ranges::replace` and `ranges::replace_if`

Section: 26.7.5 [alg.replace], 26.4 [algorithm.syn] **Status:** Immediate **Submitter:** Tim Song **Opened:** 2025-11-04 **Last modified:** 2025-11-04

Priority: Not Prioritized

View other active issues in [alg.replace].

View all other issues in [alg.replace].

Discussion:

Addresses US 159-259

The default template argument for the type of the new value in `ranges::replace` and `ranges::replace_if` should not have projections applied.

[Kona 2025-11-04; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.4 [algorithm.syn], header `<algorithm>` synopsis, as indicated:

```
[…]
  namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
             class T1 = projected_value_t<I, Proj>, class T2 = ~~T1~~iter_value_t<I>>
      requires indirectly_writable<I, const T2&> &&
               indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
      constexpr I
        replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
    template<input_range R, class Proj = identity,
             class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = ~~T1~~range_value_t<R>>
      requires indirectly_writable<iterator_t<R>, const T2&> &&
               indirect_binary_predicate<ranges::equal_to,
                                         projected<iterator_t<R>, Proj>, const T1*>
      constexpr borrowed_iterator_t<R>
        replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
             class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = ~~T1~~iter_value_t<I>>
      requires indirectly_writable<I, const T2&> &&
               indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
      I replace(Ep&& exec, I first, S last,
                const T1& old_value, const T2& new_value, Proj proj = {});  // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
             class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = ~~T1~~range_value_t<R>>
      requires indirectly_writable<iterator_t<R>, const T2&> &&
               indirect_binary_predicate<ranges::equal_to,
                                         projected<iterator_t<R>, Proj>, const T1*>
      borrowed_iterator_t<R>
        replace(Ep&& exec, R&& r, const T1& old_value, const T2& new_value,
                Proj proj = {});                                          // freestanding-deleted

    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
             class T = ~~projected~~iter_value_t<I~~, Proj~~>,
             indirect_unary_predicate<projected<I, Proj>> Pred>
      requires indirectly_writable<I, const T&>
      constexpr I replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});
    template<input_range R, class Proj = identity, class T = ~~projected_value_t<I~~range_value_t<R~~, Proj~~>,
             indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
      requires indirectly_writable<iterator_t<R>, const T&>
      constexpr borrowed_iterator_t<R>
        replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
             class Proj = identity, class T = ~~projected~~iter_value_t<I~~, Proj~~>,
             indirect_unary_predicate<projected<I, Proj>> Pred>
```

```
      requires indirectly_writable<I, const T&>
      I replace_if(Ep&& exec, I first, S last, Pred pred,
                   const T& new_value, Proj proj = {});         // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
             class T = projected_value_t<iterator_t<R>range_value_t<R, Proj>>,
             indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
      requires indirectly_writable<iterator_t<R>, const T&>
      borrowed_iterator_t<R>
        replace_if(Ep&& exec, R&& r, Pred pred, const T& new_value,
                   Proj proj = {});                             // freestanding-deleted
  }
[…]
```

2. Modify 26.7.5 [alg.replace] as indicated:

```
[…]
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         class T1 = projected_value_t<I, Proj>, class T2 = T1iter_value_t<I>>
  requires indirectly_writable<I, const T2&> &&
           indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
  constexpr I
    ranges::replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
template<input_range R, class Proj = identity,
         class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = T1range_value_t<R>>
  requires indirectly_writable<iterator_t<R>, const T2&> &&
           indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T1*>
  constexpr borrowed_iterator_t<R>
    ranges::replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = T1iter_value_t<I>>
  requires indirectly_writable<I, const T2&> &&
           indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
  I ranges::replace(Ep&& exec, I first, S last,
                    const T1& old_value, const T2& new_value, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
         class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = T1range_value_t<R>>
  requires indirectly_writable<iterator_t<R>, const T2&> &&
           indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T1*>
  borrowed_iterator_t<R>
    ranges::replace(Ep&& exec, R&& r, const T1& old_value, const T2& new_value,
                    Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         class T = projectediter_value_t<I, Proj>,
         indirect_unary_predicate<projected<I, Proj>> Pred>
  requires indirectly_writable<I, const T&>
  constexpr I ranges::replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<Irange_value_t<R, Proj>>,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
  requires indirectly_writable<iterator_t<R>, const T&>
  constexpr borrowed_iterator_t<R>
    ranges::replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, class T = projectediter_value_t<I, Proj>,
         indirect_unary_predicate<projected<I, Proj>> Pred>
  requires indirectly_writable<I, const T&>
  I ranges::replace_if(Ep&& exec, I first, S last, Pred pred,
                       const T& new_value, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
         class T = projected_value_t<iterator_t<R>range_value_t<R, Proj>>,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
  requires indirectly_writable<iterator_t<R>, const T&>
  borrowed_iterator_t<R>
    ranges::replace_if(Ep&& exec, R&& r, Pred pred, const T& new_value,
                       Proj proj = {});
```

-1- […]

4445. `sch_` must not be in moved-from state

Section: 33.13.5 [exec.task.scheduler] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

View other active issues in [exec.task.scheduler].

View all other issues in [exec.task.scheduler].

Discussion:

Addresses US 239-367

As specified, there is an implicit precondition that `sch_` is not moved from on all the member functions. If that is intended, the precondition should be made explicit.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.13.5 [exec.task.scheduler] as indicated:

```
namespace std::execution {
  class task_scheduler {
    class ts-sender;                    // exposition only

    template<receiver R>
      class state;                      // exposition only

  public:
    using scheduler_concept = scheduler_t;

    template<class Sch, class Allocator = allocator<void>>
      requires (!same_as<task_scheduler, remove_cvref_t<Sch>>)
        && scheduler<Sch>
    explicit task_scheduler(Sch&& sch, Allocator alloc = {});

    task_scheduler(const task_scheduler&) = default;
    task_scheduler& operator=(const task_scheduler&) = default;

    ts-sender schedule();

    friend bool operator==(const task_scheduler& lhs, const task_scheduler& rhs)
        noexcept;
    template<class Sch>
      requires (!same_as<task_scheduler, Sch>)
      && scheduler<Sch>
    friend bool operator==(const task_scheduler& lhs, const Sch& rhs) noexcept;

  private:
    shared_ptr<void> sch_; // exposition only
  };
}
```

4446. Bad phrasing for *SCHED*(s)

Section: 33.13.5 [exec.task.scheduler] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

View other active issues in [exec.task.scheduler].

View all other issues in [exec.task.scheduler].

Discussion:

Addresses US 240-370

`shared_ptr` owns a pointer (or `nullptr_t`), not the pointee, but *SCHED* wants the pointee.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.13.5 [exec.task.scheduler] as indicated:

> -1- `task_scheduler` is a class that models `scheduler` (33.6 [exec.sched]). Given an object `s` of type `task_scheduler`, let *SCHED* be the object pointed to by the pointer owned by `s.sch_`.

4447. Remove unnecessary `sizeof…(Env) > 1` condition

Section: 33.4 [execution.syn] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

Discussion:

Addresses US 206-325

The "`sizeof…(Env) > 1` is `true`" part seems unreachable because `CS` is ill-formed in that case.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.4 [execution.syn] as indicated:

> -2- For type `Sndr` and pack of types `Env`, let `CS` be `completion_signatures_of_t<Sndr, Env...>`. Then `single-sender-value-type<Sndr, Env...>` is ill-formed if `CS` is ill-formed ~~or if `sizeof...(Env) > 1` is `true`~~; otherwise, it is an alias for: […]

4448. Do not forward `fn` in `completion_signatures`

Section: 33.10 [exec.cmplsig] **Status:** Immediate **Submitter:** Tomasz Kamiński **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

Discussion:

Addresses US 230-360

`fn` can be called multiple times and therefore should not be forwarded.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.10 [exec.cmplsig] as indicated:

-8-

```
namespace std::execution {
  template<completion-signature... Fns>
    struct completion_signatures {
      template<class Tag>
        static constexpr size_t count-of(Tag) { return see below; }

      template<class Fn>
        static constexpr void for-each(Fn&& fn) {              // exposition only
          (~~std::forward<Fn>(fn)~~fn(static_cast<Fns*>(nullptr)), ...);
        }
    };

  […]
}
```

4449. `define_aggregate` members must be public

Section: 21.4.16 [meta.reflection.define.aggregate] **Status:** Immediate **Submitter:** Daniel Katz **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

View other active issues in [meta.reflection.define.aggregate].

View all other issues in [meta.reflection.define.aggregate].

Discussion:

The access of members of classes defined by injected declarations produced by evaluations of `std::meta::define_aggregate` is unspecified.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.4.16 [meta.reflection.define.aggregate] as indicated:

```
template<reflection_range R = initializer_list<info>>
  consteval info define_aggregate(info class_type, R&& mdescrs);
```

-7- Let C be the class represented by `class_type` and r_K be the K^{th} reflection value in `mdescrs`. […]

-8- *Constant When*: […]

-9- *Effects*: Produces an injected declaration D (7.7 [expr.const]) that defines C and has properties as follows:

(9.1) — […]
(9.2) — […]
(9.3) — […]
(9.4) — […]
(9.5) — For each r_K, there is a corresponding entity M_K with public access belonging to the class scope of D with the following properties: […]
(9.6) — […]

4450. `std::atomic_ref<T>::store_key` should be disabled for const `T`

Section: 32.5.7.3 [atomics.ref.int] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-05 **Last modified:** 2025-11-05

Priority: Not Prioritized

Discussion:

Addresses US 193-311

The new `store_key` functions modify the object, so it can't be const.

[Kona 2025-11-05; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 32.5.7.3 [atomics.ref.int], as indicated:

```
constexpr void store_key(value_type operand,
                         memory_order order = memory_order::seq_cst) const noexcept;
```

-?- *Constraints*: `is_const_v<integral-type>` is `false`.

-10- *Preconditions*: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

-11- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4 [atomics.order]).

2. Modify 32.5.7.4 [atomics.ref.float], as indicated:

```
constexpr void store_key(value_type operand,
                         memory_order order = memory_order::seq_cst) const noexcept;
```

-?- *Constraints*: `is_const_v<floating-point-type>` is `false`.

-10- *Preconditions*: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

-11- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4 [atomics.order]).

3. Modify 32.5.7.5 [atomics.ref.pointer], as indicated:

```
constexpr void store_key(see above operand,
                         memory_order order = memory_order::seq_cst) const noexcept;
```

-?- *Constraints*: `is_const_v<pointer-type>` is `false`.

-11- *Mandates*: `T` is a complete object type.

-12- *Preconditions*: `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

-13- *Effects*: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4 [atomics.order]).

4451. `make_shared` should not refer to a type `U[N]` for runtime N

Section: 20.3.2.2.7 [util.smartptr.shared.create] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-05 **Last modified:** 2025-11-06

Priority: Not Prioritized

View other active issues in [util.smartptr.shared.create].

View all other issues in [util.smartptr.shared.create].

Discussion:

Addresses US 76-139

The overloads of `make_shared` and `allocate_shared` for creating `shared_ptr<T[]>` refer to an object a type `U[N]` where `N` is a function parameter not a constant expression. Since `N` is allowed to be zero, this also allows `U[0]` which is an invalid type and so totally ill-formed.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 20.3.2.2.7 [util.smartptr.shared.create], as indicated:

```
template<class T>
  constexpr shared_ptr<T> make_shared(size_t N); // T is U[]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared(const A& a, size_t N); // T is U[]
```

-12- *Constraints*: `T` is ~~of the form `U[]`~~ an array of unknown bound.

-13- *Returns*: A `shared_ptr` to an ~~object of type `U[N]`~~ array of `N` elements of type `remove_extent_t<T>` with a default initial value~~, where `U` is `remove_extent_t<T>`~~.

-14- [*Example 2*: ...]

```
template<class T>
  constexpr shared_ptr<T> make_shared(); // T is U[N]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared(const A& a); // T is U[N]
```

-15- *Constraints*: `T` is ~~of the form `U[N]`~~ an array of known bound.

-16- *Returns*: A `shared_ptr` to an object of type `T` with a default initial value.

-17- [*Example 3*: ...]

```
template<class T>
  constexpr shared_ptr<T> make_shared(size_t N,
                                      const remove_extent_t<T>& u); // T is U[]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared(const A& a, size_t N,
                                          const remove_extent_t<T>& u); // T is U[]
```

-18- *Constraints*: `T` is ~~of the form `U[]`~~ an array of unknown bound.

-19- *Returns*: A `shared_ptr` to an ~~object of type `U[N]`~~ array of `N` elements of type `remove_extent_t<T>` where ~~`U` is `remove_extent_t<T>`~~ ~~and~~ each array element has an initial value of `u`.

-20- [*Example 4*: ...]

```
template<class T>
  constexpr shared_ptr<T> make_shared(const remove_extent_t<T>& u); // T is U[N]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared(const A& a,
                                    const remove_extent_t<T>& u); // T is U[N]
```

-21- *Constraints*: `T` is ~~of the form `U[N]`~~ an array of known bound.

-22- *Returns*: A `shared_ptr` to an object of type `T`, where each array element of type `remove_extent_t<T>` has an initial value of `u`.

-23- [*Example 5*: ...]

```
template<class T>
  constexpr shared_ptr<T> make_shared_for_overwrite(); // T is U[N]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a); // T is U[N]
```

-24- *Constraints*: `T` is not an array of unknown bound.

-25- *Returns*: A `shared_ptr` to an object of type `T`.

-26- [*Example 6*: ...]

```
template<class T>
  constexpr shared_ptr<T> make_shared_for_overwrite(size_t N); // T is U[]
template<class T, class A>
  constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a, size_t N); // T is U[]
```

-27- *Constraints*: `T` is an array of unknown bound.

-28- *Returns*: A `shared_ptr` to an ~~object of type `U[N]`~~ array of `N` elements of type `remove_extent_t<T>` ~~, where `U` is `remove_extent_t<T>`~~.

-29- [*Example 7*: ...]

4452. Make *deref-move* constexpr

Section: 26.11.1 [specialized.algorithms.general] **Status:** Immediate **Submitter:** S.B. Tam **Opened:** 2025-11-05 **Last modified:** 2025-11-06

Priority: Not Prioritized

View all other issues in [specialized.algorithms.general].

Discussion:

`std::uninitialized_move` and `std::uninitialized_move_n` are constexpr and invoke *`deref-move`*, but *`deref-move`* is not constexpr. This looks like an obvious mistake.

Jiang An pointed out that P3508R0 and LWG 3918[(i)], both touching `std::uninitialized_move(_n)`, were adopted at the same meeting, and unfortunately none of them was aware of the other.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.11.1 [specialized.algorithms.general], as indicated:

```
template<class I>
  constexpr decltype(auto) deref-move(I& it) {
```

```
    if constexpr (is_lvalue_reference_v<decltype(*it)>)
      return std::move(*it);
    else
      return *it;
  }
```

4455. Add missing constraint to *basic-sender*::get_completion_signatures definition

Section: 33.9.2 [exec.snd.expos] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-06 **Last modified:** 2025-11-06

Priority: Not Prioritized

View other active issues in [exec.snd.expos].

View all other issues in [exec.snd.expos].

Discussion:

Addresses US 215-356

The definition of *basic-sender*::get_completion_signatures is missing the *decays-to*<*basic-sender*> type constraint.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.2 [exec.snd.expos], as indicated (just above paragraph 43):

```
template<class Tag, class Data, class... Child>
  template<~~class~~ decays-to<basic-sender> Sndr, class... Env>
    constexpr auto basic-sender<Tag, Data, Child...>::get_completion_signatures();
```

4456. Decay Data and Child in *make-sender*

Section: 33.9.2 [exec.snd.expos] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-06 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [exec.snd.expos].

View all other issues in [exec.snd.expos].

Discussion:

Addresses US 211-351

The *Mandates*: for *make-sender* defines Sndr as a type that is different from what the *Returns*: element uses.

[Kona 2025-11-06; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.2 [exec.snd.expos], as indicated:

```
template<class Tag, class Data = see below, class... Child>
  constexpr auto make-sender(Tag tag, Data&& data, Child&&... child);
```

-24- *Mandates*: The following expressions are true:

(24.1) — semiregular<Tag>
(24.2) — *movable-value*<Data>
(24.3) — (sender<Child> && ...)
(24.4) — dependent_sender<Sndr> || sender_in<Sndr>, where Sndr is *basic-sender*<Tag, ~~Data, Child~~ decay_t<Data>, decay_t<Child>...> as defined below.

> *Recommended practice*: If evaluation of sender_in<Sndr> results in an uncaught exception from the evaluation of get_completion_signatures<Sndr>(), the implementation should include information about that exception in the resulting diagnostic.

-25- *Returns*: A prvalue of type *basic-sender*<Tag, decay_t<Data>, decay_t<Child>...> that has been direct-list-initialized with the forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below.

4459. Protect get_completion_signatures fold expression from overloaded commas

Section: 33.9.2 [exec.snd.expos] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-07 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [exec.snd.expos].

View all other issues in [exec.snd.expos].

Discussion:

Addresses US 214-355

The fold expression can pick up overloaded comma operators.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.2 [exec.snd.expos], as indicated:

```
template<class Sndr, class... Env>
  static consteval void default-impls::check-types();
```

-40- Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by *`indices-for`*`<Sndr>`.

-41- *Effects*: Equivalent to: `((void)get_completion_signatures<`*`child-type`*`<Sndr, Is>, `*`FWD-ENV-T`*`(Env)...>(), ...);`

4461. *`stop-when`* needs to evaluate unstoppable tokens

Section: 33.9.12.17 [exec.stop.when] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-06 **Last modified:** 2025-11-07

Priority: Not Prioritized

Discussion:

Addresses US 226-345

For the case where `token` models `unstoppable_token` the expression that *`stop-when`* is expression-equivalent to needs to include the fact that `token` is evaluated (even if not used).

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 33.9.12.17 [exec.stop.when], as indicated:

-2- The name *`stop-when`* denotes an exposition-only sender adaptor. For subexpressions `sndr` and `token`:

(2.1) — If `decltype((sndr))` does not satisfy `sender`, or `remove_cvref_t<decltype((token))>` does not satisfy `stoppable_token`, then *`stop-when`*`(sndr, token)` is ill-formed.
(2.2) — Otherwise, if `remove_cvref_t<decltype((token))>` models `unstoppable_token` then *`stop-when`*`(sndr, token)` is expression-equivalent to `(void)token, sndr`, except that `token` and `sndr` are indeterminately sequenced.

4462. Algorithm requirements don't describe semantics of `s - i` well

Section: 26.2 [algorithms.requirements] **Status:** Immediate **Submitter:** Jonathan Wakely **Opened:** 2025-11-07 **Last modified:** 2025-11-07

Priority: Not Prioritized

View other active issues in [algorithms.requirements].

View all other issues in [algorithms.requirements].

Discussion:

Addresses US 154-252

"the semantics of `s - i` has" is not grammatically correct. Additionally, "type, value, and value category" are properties of expressions, not "semantics".

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.2 [algorithms.requirements], as indicated:

-11- In the description of the algorithms, operator + is used for some of the iterator categories for which it does not have to be defined. In these cases the semantics of `a + n` are the same as those of

```
auto tmp = a;
for (; n < 0; ++n) --tmp;
```

```
for (; n > 0; --n) ++tmp;
return tmp;
```

Similarly, operator – is used for some combinations of iterators and sentinel types for which it does not have to be defined. If [a, b) denotes a range, the semantics of `b - a` in these cases are the same as those of

```
iter_difference_t<decltype(a)> n = 0;
for (auto tmp = a; tmp != b; ++tmp) ++n;
return n;
```

and if [b, a) denotes a range, the same as those of

```
iter_difference_t<decltype(b)> n = 0;
for (auto tmp = b; tmp != a; ++tmp) --n;
return n;
```

For each iterator `i` and sentinel `s` produced from a range `r`, the semantics of `s - i` are the same as those of an expression that has the same type, value, and value category as `ranges::distance(i, s)`.

[*Note 3*: The implementation can use `ranges::distance(r)` when that produces the same value as `ranges::distance(i, s)`. This can be more efficient for sized ranges. — *end note*]

4463. Change wording to 'model' from 'subsumes' in [algorithms.parallel.user]

Section: 26.3.2 [algorithms.parallel.user] **Status:** Immediate **Submitter:** Ruslan Arutyunyan **Opened:** 2025-11-07 **Last modified:** 2025-11-07

Priority: Not Prioritized

Discussion:

Addresses US 155-253

"Subsumes" word does not work here because `regular_invocable` and `invocable` subsume each other.

Proposed change: Say that the type is required to model `regular_invocable`.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.3.2 [algorithms.parallel.user], as indicated:

> -1- Unless otherwise specified, invocable objects passed into parallel algorithms as objects of a type denoted by a template parameter named `Predicate`, `BinaryPredicate`, `Compare`, `UnaryOperation`, `BinaryOperation`, `BinaryOperation1`, `BinaryOperation2`, `BinaryDivideOp`, or constrained by a concept whose semantic requirements include that ~~subsumes~~ the type models `regular_invocable` and the operators used by the analogous overloads to these parallel algorithms that are formed by an invocation with the specified default predicate or operation (where applicable) shall not directly or indirectly modify objects via their arguments, nor shall they rely on the identity of the provided objects.

4464. §[alg.merge] Wording tweaks

Section: 26.8.6 [alg.merge] **Status:** Immediate **Submitter:** Ruslan Arutyunyan **Opened:** 2025-11-07 **Last modified:** 2025-11-08

Priority: Not Prioritized

View other active issues in [alg.merge].

View all other issues in [alg.merge].

Discussion:

Addresses US 163-262

The original text of the "US 163-262" issue says: "Bullets 1.3 and 1.4 and paragraph 3 should say *`E`*`(e1, e2)` instead of *`E`*`(e1, e1)`" in [alg.merge]. The problem, though, was introduced when merging P3179R9 "Parallel Range Algorithms" proposal. The original wording of P3179 does not have parentheses after *`E`*. Those extra parameters in *`E`* do not bring clarity to `merge` algorithm. The proposed resolution is to strike them through.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.8.6 [alg.merge], as indicated:

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  constexpr OutputIterator
    merge(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, InputIterator2 last2,
          OutputIterator result);
[…]
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
         sized-random-access-range OutR, class Comp = ranges::less,
```

```
      class Proj1 = identity, class Proj2 = identity>
  requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
  ranges::merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, borrowed_iterator_t<OutR>>
    ranges::merge(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
                  Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

-1- Let:

(1.1) — *N* be: […]
(1.2) — `comp` be `less{}`, `proj1` be `identity{}`, and `proj2` be `identity{}`, for the overloads with no parameters by those names;
(1.3) — *E*~~`(e1, e1)`~~ be `bool(invoke(comp, invoke(proj2, e2), invoke(proj1, e1)))`;
(1.4) — *K* be the smallest integer in `[0, last1 - first1)` such that for the element `e1` in the position `first1 + K` there are at least `N - K` elements `e2` in `[first2, last2)` for which *E*~~`(e1, e1)`~~ holds, and be equal to `last1 - first1` if no such integer exists.

-2- *Preconditions*: The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

-3- *Effects*: Copies the first *K* elements of the range `[first1, last1)` and the first *N - K* elements of the range `[first2, last2)` into the range `[result, result + N)`. If an element `a` precedes `b` in an input range, `a` is copied into the output range before `b`. If `e1` is an element of `[first1, last1)` and `e2` of `[first2, last2)`, `e2` is copied into the output range before `e1` if and only if *E*~~`(e1, e1)`~~ is `true`.

4465. §[alg.partitions] Clarify *Returns:* element

Section: 26.8.5 [alg.partitions] **Status:** Immediate **Submitter:** Ruslan Arutyunyan **Opened:** 2025-11-07 **Last modified:** 2025-11-08

Priority: Not Prioritized

View all other issues in [alg.partitions].

Discussion:

Addresses US 162-261

In 26.8.5 [alg.partitions] p21 the wording is unclear what happens if there is no such element. The proposed resolution tries to clarify that without complicating the wording. If the proposed resolution (or something along those lines) fails, the recommendation is to reject US 162-261.

[Kona 2025-11-07; approved by LWG. Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.8.5 [alg.partitions], as indicated:

```
template<class InputIterator, class OutputIterator1, class OutputIterator2, class Predicate>
  constexpr pair<OutputIterator1, OutputIterator2>
    partition_copy(InputIterator first, InputIterator last,
                   OutputIterator1 out_true, OutputIterator2 out_false, Predicate pred);
[…]
template<execution-policy Ep, sized-random-access-range R,
         sized-random-access-range OutR1, sized-random-access-range OutR2,
         class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR1>> &&
           indirectly_copyable<iterator_t<R>, iterator_t<OutR2>>
  ranges::partition_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR1>,
                                borrowed_iterator_t<OutR2>>
    ranges::partition_copy(Ep&& exec, R&& r, OutR1&& out_true_r, OutR2&& out_false_r,
                           Pred pred, Proj proj = {});
```

-14- Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let *E*`(x)` be `bool(invoke(pred, invoke(proj, x)))`.

[…]

-19- *Preconditions*: The input range and output ranges do not overlap. […]

-20- *Effects*: For each iterator `i` in `[first, first + N)`, copies `*i` to the output range `[out_true, last_true)` if *E*`(*i)` is `true`, or to the output range `[out_false, last_false)` otherwise.

-21- *Returns*: Let ~~*o1*~~ *Q* be the ~~iterator past the last~~ number of elements copied ~~element~~ into the output range `[out_true, last_true)`, and ~~*o2*~~ *V* be the ~~iterator past the last~~ number of elements copied ~~element~~ into the output range `[out_false, last_false)`. Returns:

(21.1) — {~~*o1*~~`out_true + Q`, ~~*o2*~~`out_false + V`} for the overloads in namespace `std`.
(21.2) — `{first + N,` ~~*o1*~~`out_true + Q,` ~~*o2*~~`out_false + V}` for the overloads in namespace `ranges`.

-22- *Complexity*: At `most last - first` applications of `pred` and `proj`.